

UNIVERSITY OF PLYMOUTH

MAL3018

Computing Project

LOOM – Implementation of a Modular Node-Based Visual
Workflow

BSSE2409163

Acknowledgements

I would like to thank my supervisor, Eric Kong Kok Wah, for their guidance and valuable feedback throughout this project. My gratitude also goes to my friends and peers who provided support, insights, and encouragement during the development process. I am thankful for the resources and facilities provided by University of Plymouth and Peninsula College Malaysia, which made this work possible. Finally, to my family, for their patience and support. Without them, I might have accidentally debugged my life instead of my code.

Abstract

Programming can be daunting for beginners, especially when lines of code turn into spaghetti and data flows remain invisible. LOOM tackles this challenge head-on by turning Python logic into an interactive, visual experience. Designed for students who want to understand how systems work, not just how to type syntax, LOOM uses a three-layer "Face-Brain-Heart" architecture: a React and React Flow frontend for intuitive interaction, a FastAPI backend for state management, and a standalone Python engine for execution. Its standout features include a dynamic node loader for custom scripts, a dependency-based scheduler for predictable flows, and real-time WebSocket feedback which make experimentation seamless and modularity natural. While the prototype still has room to polish UI and scale to larger workflows, it proves that a flow-first visual approach can make computational logic approachable, flexible, and even fun for learners.

Table of Contents

Acknowledgements.....	i
Abstract	ii
Chapter 1: Introduction	7
1.1 Project Overview	7
1.2 Problem Statement	8
1.3 Project Aim and Objectives	8
1.4 Scope of the Project	8
In-Scope	8
Out-of-Scope.....	8
1.5 Significance of the Project	9
1.6 Structure of the Report	9
Chapter 2: Background, Objectives, and Deliverables	10
2.1 Background	10
Context of the Problem	10
Motivation	10
Existing Solutions / Systems	11
User / Stakeholder Input	11
2.2 Objectives	12
2.3 Deliverables.....	12
Internal Deliverables	12
Final Deliverables.....	12
Chapter 3: Method of Approach	13
3.1 Risk Assessment.....	13
3.2 Requirements Analysis	14
Functional Requirements	14
Non-Functional Requirements	14
User Stories	14
Minimum Viable Product (MVP)	15
3.3 Technologies Used	15
3.4 Best Practices	16
3.5 Testing Strategy	16

3.6 System Design	17
Three-Layer Architectural Design.....	17
Data Propagation Model	17
3.7 Assumptions and Constraints	18
Chapter 4: Legal, Social and Professional Issues	19
4.1 Social Issues	19
4.2 Legal Issues	21
4.3 Professional Issues	22
Chapter 5: Project Management.....	24
5.1 Project Management Tools	24
5.2 Methodology.....	25
Hybrid Approach (Design-First + Agile).....	25
Sprint Structure	26
5.3 Monitoring and Control.....	27
Chapter 6: Project Stages / Development Phases	28
6.1 Phase 1: Planning & Preparation	28
6.2 Phase 2: Design & Setup	29
6.3 Phase 3: Implementation	30
Dynamic Node Loader	31
Dependency-Based Scheduler.....	32
Frontend Integration	33
Logging.....	34
WebSocket Communication	35
Additional Features and Enhancements	36
6.4 Phase 4: Testing & Refinement	37
Logging & Debugging Refinements.....	37
Design Decisions for Consistency	37
6.5 Phase 5: Finalisation	38
6.6 Reflection and Lessons Learned.....	39
Chapter 7: Output Documentation	41
7.1 Output Diagrams.....	41
Architecture Diagram	41

Activity Diagram	43
Sequence Diagram.....	44
7.2 Database Structure	45
7.3 Completed Requirements.....	47
MVP Features Completed	47
Post-MVP Features (Implemented)	47
Post-MVP Features (Future Work)	47
Objective Completion	47
7.4 API and Command/Sync Mapping	48
COMMAND MAP	49
SYNC MAP	50
7.5 User Testing Results	51
Testing Description.....	51
Results Summary.....	51
Key Findings.....	51
User Feedback Highlights	51
Conclusion	51
Chapter 8: End-Project Evaluation.....	55
8.1 Design Adherence	55
8.2 Objective Achievement	56
8.3 System Evaluation.....	57
8.4 Future Enhancements	58
Chapter 9: Post-Mortem / Critical Reflection	59
9.1 Technology Review	59
9.2 Project Management Reflection	59
9.3 Personal Performance	60
9.4 User Testing Reflection	60
9.5 Ethical and Accessibility Reflection	61
9.6 Overall Reflection.....	61
Chapter 10: Conclusion.....	62
10.1 Project Summary.....	62
10.2 Achievement of Aim and Objectives.....	62

10.3 Contribution of the Project	62
10.4 Final Remarks	63
Reference.....	64
Appendix	65

Word Count : 9461

Chapter 1: Introduction

1.1 Project Overview

In the early stages of learning computer science, students often face difficulty when moving from understanding basic syntax to designing complete systems. While they may understand programming concepts such as loops and conditionals, many struggle to understand how data flows between different parts of a program. This makes it hard to see the “big picture” of how a system works.

Most programming learning methods focus on writing code line-by-line. However, real-world systems are not linear they are made up of multiple connected components working together.

To address this issue, this project introduces **LOOM**, a visual orchestration framework. The name is inspired by a weaving loom, where threads are combined to form a complete structure. In this system, Python scripts act as “threads,” while the LOOM engine connects them into a visual and functional flow.

LOOM is designed as a learning support tool to help students shift their focus from writing individual lines of code to understanding overall system structure. This is especially important in today’s environment where code can be generated automatically, but system design still requires human understanding.

Target Users:

- Beginner to intermediate computer science students
- Learners who struggle with understanding program flow and architecture

Github	https://github.com/TanggaPendek/Loom
Youtube	https://youtu.be/XIzYka-iZ1Y
E-Portfolio	https://v3.pebblepad.co.uk/spa/#/public/mtw79Z8zwmw669768mZgs6nz4M

1.2 Problem Statement

Several challenges exist in current programming education:

1. **Poor Code Structure (“Spaghetti Code”)**

Beginners often write long, unstructured programs. This makes the code difficult to understand, debug, and maintain.

2. **Lack of Visual Understanding**

Students find it difficult to imagine how data moves through a program. Reading code does not always clearly show how different parts interact during execution.

3. **Focus on Syntax Over Design**

Many tools and courses focus more on how to write code rather than how to design systems. This leaves students unprepared for more complex software development tasks.

These issues highlight the need for a tool that supports both understanding and visualization of program structure.

1.3 Project Aim and Objectives

Aim:

To develop a visual orchestration platform that helps students understand software architecture by representing Python logic through an interactive graphical interface.

Objectives:

- To shift focus from code syntax to overall data flow and system structure
- To provide a visual graph-based interface for representing program execution
- To encourage modular design using independent and reusable nodes

1.4 Scope of the Project

In-Scope

- Graph-based interface using React and ReactFlow
- Python-based backend with a queue-based execution system
- Hybrid desktop application bridge using pywebview
- Basic node modules (e.g., math operations and simple input/output) for testing

Out-of-Scope

- Advanced parallel processing (execution follows dependency order only)
- Complex media processing such as 3D rendering or video handling

1.5 Significance of the Project

This project provides several benefits:

- **For Students:**
Helps improve understanding of program flow and system design
- **For Education:**
Supports teaching methods that go beyond syntax and focus on structure
- **For Future Development:**
Encourages better coding practices such as modularity and clean design

1.6 Structure of the Report

This report consists of 10 chapters:

- **Chapter 1:** Introduction
- **Chapter 2:** Background, Objectives, and Deliverables
- **Chapter 3:** Method of Approach
- **Chapter 4:** Legal, Social and Professional Issues
- **Chapter 5:** Project Management
- **Chapter 6:** Project Stages / Development Phases
- **Chapter 7:** Output Documentation
- **Chapter 8:** End-Project Evaluation
- **Chapter 9:** Post-Mortem / Critical Reflection
- **Chapter 10:** Conclusion

Chapter 2: Background, Objectives, and Deliverables

2.1 Background

Context of the Problem

Programming is a fundamental skill in modern computing fields, yet many beginners struggle with understanding abstract concepts such as syntax, control flow, and data handling. Traditional text-based programming environments often create a steep learning curve, especially for users with little to no prior experience.

In industry, there has been a growing trend toward **visual programming systems** that simplify complex workflows by representing logic as interconnected components. These systems allow users to focus on logic rather than syntax, making development more intuitive and accessible.

Examples of such systems include:

- **Blender Geometry Nodes** – allows users to generate complex 3D structures through node-based logic rather than manual modelling.
- **Node-RED** – widely used in IoT applications to visually connect devices, APIs, and services.
- **n8n** – focuses on workflow automation, enabling users to define triggers and data flow between services.
- **Rete.js** – a framework designed to build node-based editors, separating UI representation from execution logic.

These tools demonstrate that **flow-based and visual programming approaches** can effectively reduce complexity and improve usability.

Motivation

The motivation behind this project comes from the difficulty beginners face when learning programming concepts through traditional coding methods. Syntax errors, lack of visualization, and abstract execution flow often discourage new learners.

By observing existing visual systems, it is evident that representing logic visually can significantly improve understanding and engagement. Therefore, this project aims to bridge the gap between learning programming concepts and applying them in a practical environment.

Additionally, this project is inspired by the idea of making programming feel more like building or assembling, rather than writing rigid lines of code.

Existing Solutions / Systems

While existing tools provide strong foundations, they also have limitations:

Tool	Strength	Limitation
Blender Geometry Nodes	Powerful node-based system for procedural and visual workflows	Primarily designed for 3D content, not general programming logic
Node-RED	Excellent for IoT, APIs, and system integrations with visual flows	Limited support for structured programming concepts and complex logic design
n8n	Strong automation platform with easy integration and workflow management	Less flexible for building custom, low-level programming logic
Rete.js	Highly flexible framework for building custom node-based editors	Requires significant technical knowledge and development effort

These limitations highlight the need for a system specifically designed for learning and experimenting with programming logic in a simplified and modular way.

User / Stakeholder Input

Initial feedback was gathered through informal discussions with peers and students in computing-related fields. The following key issues were identified:

- Difficulty understanding program flow
- Confusion caused by syntax errors
- Lack of engaging tools for beginners

This feedback supports the need for a visual-based programming approach that improves clarity and usability.

2.2 Objectives

The objectives of this project are defined based on the identified problems:

1. **To simplify programming workflow**
Develop a drag-and-drop system that reduces dependency on syntax and allows users to focus on logical structure.
2. **To improve understandability**
Represent program execution using a visual graph to clearly show the flow of data and operations.
3. **To enhance modularity**
Enable users to create and reuse small functional components ("nodes") across different workflows.

2.3 Deliverables

Internal Deliverables

These are artifacts produced during development:

- System design diagrams (architecture, flowcharts)
- Prototype interfaces and wireframes
- Development documentation and planning materials

Final Deliverables

The final outputs of the project include:

- A working web-based application (React, FastAPI, Python engine)
- A "NodeBank" containing reusable Python-based nodes
- User documentation and guide for system usage

Chapter 3: Method of Approach

3.1 Risk Assessment

The development of the LOOM system involves several risks across technical complexity, time constraints, and knowledge limitations.

From a technical perspective, the primary risk lies in synchronisation between the frontend (React), backend (FastAPI), and execution engine (Python). Misalignment between these components may result in incorrect graph execution or data inconsistency. Additionally, the implementation of a signal-based communication system and logging system, introduces uncertainty due to limited understanding of their internal mechanisms.

Time-related risks are also present, as the project requires integration of multiple layers within a limited development period. Delays in debugging or integration may affect overall progress.

Furthermore, the learning curve associated with graph-based systems, asynchronous communication, and system architecture design may slow down development.

To mitigate these risks, a modular development approach is adopted, allowing each layer (frontend, backend, execution engine) to be developed and tested independently. Frequent manual testing and incremental feature implementation are used to detect issues early. Additionally, reliance on simple data formats such as JSON helps reduce integration complexity.

3.2 Requirements Analysis

Functional Requirements

The system is required to support the following functionalities:

- Users can create, delete, and modify nodes within a visual graph interface
- Users can connect nodes to define execution flow
- The system processes graph-based workflows upon user execution
- Data is propagated between nodes based on connections
- Users can save and load projects using JSON format
- Backend handles indexing and persistence of project data
- Execution engine processes node operations such as mathematical and simple input/output tasks

Non-Functional Requirements

The system must also satisfy several quality attributes:

- **Usability:** The interface should be intuitive, allowing users to interact with nodes through drag-and-drop operations
- **Performance:** The system should execute small to medium graphs with minimal delay
- **Reliability:** Data should not be lost during execution or saving processes
- **Maintainability:** Code should be modular to allow future expansion
- **Scalability:** The architecture should allow extension to more complex node types and workflows
- **Transparency:** Execution results should be clearly displayed via the logging system

User Stories

- As a student, I want to visually build workflows so that I can understand how data flows between operations
- As a teacher, I want to demonstrate computational logic using a graphical interface so that concepts are easier to explain
- As a beginner programmer, I want to execute logic without writing full code so that I can focus on understanding the process
- As a user, I want to save and load my workflows so that I can continue my work later

Minimum Viable Product (MVP)

The MVP of the LOOM system includes full integration of the three-layer architecture:

- Frontend supports node creation, drag-and-drop, and graph visualisation
- Backend supports file saving, loading, and indexing
- Execution engine processes graph workflows
- Basic node types (math and simple input/output) are supported
- Data propagation between nodes is functional

3.3 Technologies Used

The system is developed using a combination of modern web and backend technologies.

- **Frontend:** React with React Flow
React is chosen for its component-based architecture, allowing efficient UI management. React Flow is used to implement the node-based graph interface due to its built-in support for node connections and visualisation.
- **Backend:** FastAPI (Python)
FastAPI is selected for its lightweight and high-performance capabilities, particularly in handling asynchronous operations and API communication.
- **Execution Engine:** Python
Python is used for executing node logic due to its simplicity and strong support for computational tasks.
- **Data Format:** JSON
JSON is used for storing and transferring graph data due to its simplicity and compatibility across all system layers.
- **Communication:** WebSocket
Used to provide lightweight, real-time communication between frontend and backend for execution state updates
- **Additional Systems:**
A signal-based communication system is implemented to decouple components, avoiding direct dependencies between nodes.
A logging system is also used to display execution results on the frontend, improving user feedback and system transparency

3.4 Best Practices

Several best practices are followed during development to ensure code quality and maintainability.

- **Modular Architecture:**
The system is divided into three independent layers (frontend, backend, execution engine)
- **Code Organisation:**
Components and logic are separated based on functionality
- **Version Control:**
Git is used to manage development progress and track changes
- **Naming Conventions:**
Consistent naming is applied across variables, functions, and components
- **Reusability:**
Node components and backend utilities are designed to be reusable
- **Decoupling:**
The signal system reduces direct dependencies between components
- **User Feedback Design:**
Visual indicators such as overlays are used to communicate system state during execution

3.5 Testing Strategy

Due to time constraints and project scope, testing is primarily conducted through manual and peer-based approaches.

- **Manual Testing:**
The system is tested by interacting with the interface, creating graphs, and executing workflows to verify expected behaviour
- **Integration Testing:**
Basic integration testing is performed by checking communication between frontend, backend, and execution engine
- **User Acceptance Testing:**
Feedback from peers is used to evaluate usability and functionality
- **Automated Testing:**
Automated unit and integration testing are not implemented in this project

Although testing is limited, frequent testing during development helps identify and resolve issues early.

3.6 System Design

Three-Layer Architectural Design

The LOOM system adopts a decoupled three-layer architecture, referred to as the "Face–Brain–Heart" model.

- The **Frontend (Face)** is implemented using React and React Flow. It manages the graphical interface, including node creation, connection, and user interaction.
- The **Backend (Brain)** is implemented using FastAPI. It is responsible for managing project data, handling persistence, and converting the graph structure into a structured JSON execution map.
- The **Engine (Heart)** operates as an independent Python process. It executes node operations and processes data flow without directly affecting the frontend or backend.

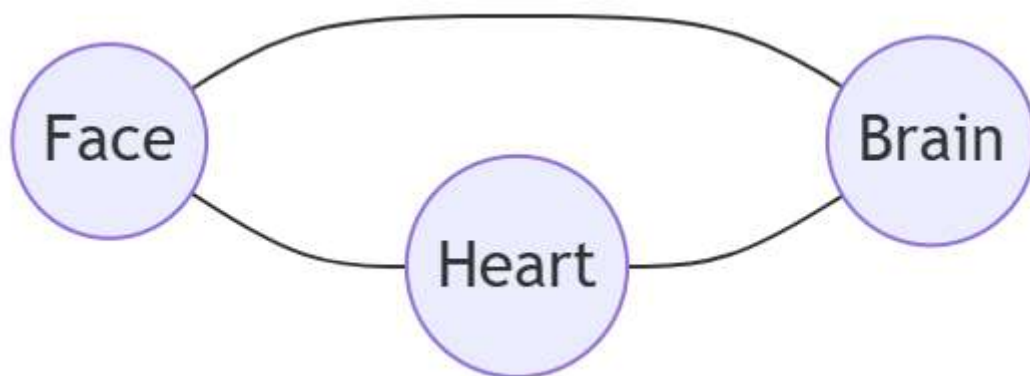


Figure 1: Loom System Design

This separation improves maintainability and allows each layer to function independently.

Data Propagation Model

The system uses a push-based data propagation model.

When a node completes execution, its output is sent to all connected nodes as input. This allows data to flow dynamically through the graph structure.

This model reflects real-world systems where data is passed between services through pipelines, enabling flexible and scalable workflow execution.

3.7 Assumptions and Constraints

Assumptions

- Users have basic familiarity with graphical user interfaces
- The system runs in an environment that supports modern web technologies and Python
- Graphs created are relatively small and manageable

Constraints

- Limited development time restricts full implementation of advanced features
- Testing is limited to manual methods
- Performance may decrease with very large or complex graphs
- WebSocket usage is limited to execution state indication and does not support full real-time data streaming

Chapter 4: Legal, Social and Professional Issues

4.1 Social Issues

LOOM is developed as a visual, graph-based programming tool aimed primarily at students and beginners. Its design simplifies traditional coding by allowing users to construct logic through interconnected nodes rather than writing syntax manually. This approach shifts the focus from memorization of programming language rules to understanding logical structure and data flow, which is often a major barrier for new learners.

User Interaction Implications

By reducing reliance on syntax, LOOM lowers the barrier to entry for programming and enables users to engage with computational thinking at an earlier stage. Users can experiment more freely, observe how data moves between nodes, and quickly identify logical errors through visual representation. This promotes exploratory learning and can increase confidence among beginners who might otherwise feel intimidated by traditional programming environments.

However, this abstraction introduces a trade-off. Users may become reliant on the visual interface and struggle when transitioning to text-based programming languages such as Python or Java. In conventional environments, syntax, debugging, and code structuring are essential skills that are not directly reinforced within LOOM. As a result, learners might develop an incomplete understanding of programming concepts if the tool is used in isolation.

To mitigate this issue, LOOM should ideally be positioned as a complementary learning tool rather than a replacement for traditional coding. Integrating features such as optional code previews or hybrid modes could help bridge the gap between visual and text-based programming, supporting a smoother learning progression.

Fairness, Usability, and Accessibility

LOOM promotes usability through a simplified and intuitive interface, making programming more approachable for users with little or no technical background. The drag-and-drop interaction model reduces cognitive load and allows users to focus on problem-solving rather than syntax correctness.

Despite these advantages, certain accessibility limitations exist:

- Graph-based interfaces may be challenging for users with visual impairments, particularly those who rely on screen readers
- Dense node layouts can become difficult to navigate and interpret as project complexity increases
- Performance may vary depending on device capability, potentially disadvantaging users with lower-end hardware

To maintain fairness and inclusivity, the system should aim for clear interface design, consistent visual hierarchy, and readable layouts. Features such as zooming, node grouping, and adjustable contrast can improve usability for a wider range of users. Future improvements may also include keyboard navigation, screen reader compatibility, and alternative representations of workflow

Impact on Different User Groups

LOOM affects different user groups in distinct ways:

- Students benefit from simplified learning, interactive experimentation, and clearer visualization of logical processes. This can improve engagement and retention of fundamental programming concepts.
- Educators can leverage structured NodeBanks to guide learning, standardize exercises, and demonstrate concepts in a controlled and visual manner.
- Advanced users may find the system restrictive, particularly when implementing complex logic, optimizing performance, or working with large-scale systems. The abstraction that benefits beginners may become a limitation at higher levels of expertise.

Overall, LOOM is most effective when targeted toward early-stage learners and educational environments, where its strengths in visualization and accessibility can be fully utilized.

4.2 Legal Issues

Data Protection and Privacy

LOOM operates as a local application, meaning that user data and executed scripts remain on the user's device. This design choice significantly reduces risks associated with data transmission, unauthorized access, and third-party data storage. It also aligns with general principles of data protection regulations such as Malaysia's Personal Data Protection Act (PDPA), which emphasizes user consent, data minimization, and secure handling of personal information.

Since no personal data is collected, processed, or stored externally, the system presents minimal direct privacy risks. However, indirect risks may still arise through the execution of user-defined scripts. For example, a script could potentially access local files, modify system data, or expose sensitive information if not properly controlled.

As such, users must be made aware of the potential risks associated with executing untrusted or unknown scripts. Transparency regarding system capabilities and limitations is essential to ensure informed usage.

Intellectual Property

LOOM enables users to create, modify, and reuse node-based logic structures, including custom NodeBanks. These user-generated components are considered the intellectual property of their creators. Any distribution or sharing of such components should respect ownership rights and include appropriate attribution where necessary.

Additionally, if LOOM is extended to support sharing platforms or collaborative environments in the future, mechanisms for licensing, version control, and ownership tracking may be required. This ensures that creators retain recognition for their work while allowing others to build upon it in a fair and structured manner.

Terms of Service and Consent

At its current stage, LOOM does not implement a formal terms-of-service agreement or user consent mechanism. However, it is recommended that such provisions be introduced in future iterations to clearly define acceptable usage and user responsibilities.

Users should be informed about:

- The risks associated with executing custom or third-party scripts
- The absence of built-in security protections in the current prototype
- Acceptable and intended usage within an educational context

Providing clear documentation or user guidelines can help ensure that users understand the scope and limitations of the system, reducing the likelihood of misuse or unintended consequences.

4.3 Professional Issues

Ethical Considerations

The development of LOOM reflects key professional and ethical principles, including promoting accessibility, reducing barriers to education, and supporting inclusive learning environments. By simplifying programming concepts, the system enables a wider audience to engage with computational thinking.

However, ethical responsibility extends beyond accessibility. Developers must also consider the potential risks associated with system usage, particularly when users may lack the technical knowledge to identify or mitigate those risks. Ensuring that users are not exposed to unnecessary harm whether through misleading design, lack of safeguards, or unclear system behaviour is a critical aspect of responsible development.

Security Considerations

A significant professional concern in LOOM is the execution of user-defined Python scripts. While this feature provides flexibility and extensibility, it also introduces potential security risks. Malicious or poorly written scripts may perform harmful actions, such as deleting files, consuming excessive system resources, or accessing sensitive data.

At the current stage of development, no dedicated security mechanisms such as sandboxing, execution isolation, or permission controls have been implemented. This limitation is acknowledged as part of the system's prototype scope.

Despite this, the risks associated with script execution are non-trivial and must be clearly communicated to users. In educational settings, this risk can be mitigated by restricting usage to trusted NodeBanks curated by instructors or system administrators.

Future improvements should prioritize security enhancements, including:

- Implementation of execution sandboxing to restrict access to system resources
- Warning prompts or confirmation dialogs before running scripts
- Validation and verification of imported nodes
- Logging and monitoring of script execution for debugging and accountability

These measures would significantly improve system safety and align LOOM with standard software security practices.

Responsible Software Development and Licensing

LOOM is developed using modern technologies such as React, FastAPI, and Python. The use of third-party frameworks requires compliance with their respective licenses, including proper attribution and adherence to usage terms. Failure to comply with these licenses could result in legal and ethical issues.

As a prototype, LOOM prioritizes functionality and concept validation over production-level robustness. However, responsible software development requires transparency about system limitations, including known risks and unimplemented features. Clearly communicating these limitations ensures that users and stakeholders have realistic expectations of the system.

Chapter 5: Project Management

5.1 Project Management Tools

The project was managed using a lightweight toolset appropriate for a single-developer environment. Version control was handled using GitHub, which enabled systematic tracking of code changes, feature development, and version history throughout the project lifecycle.

GitHub also served as a central reference point for project progression, allowing incremental updates and rollback capability when required. This was particularly useful during the integration phase, where frequent adjustments were made between the frontend and backend components.

Task tracking and prioritisation were managed using a simple, self-maintained logging approach. Key development goals and milestones were recorded periodically to monitor progress and ensure alignment with the overall project timeline.

Due to the limited scale and individual nature of the project, formal project management tools such as Jira or Trello were not adopted. This approach allowed greater flexibility while maintaining sufficient structure for effective progress tracking.

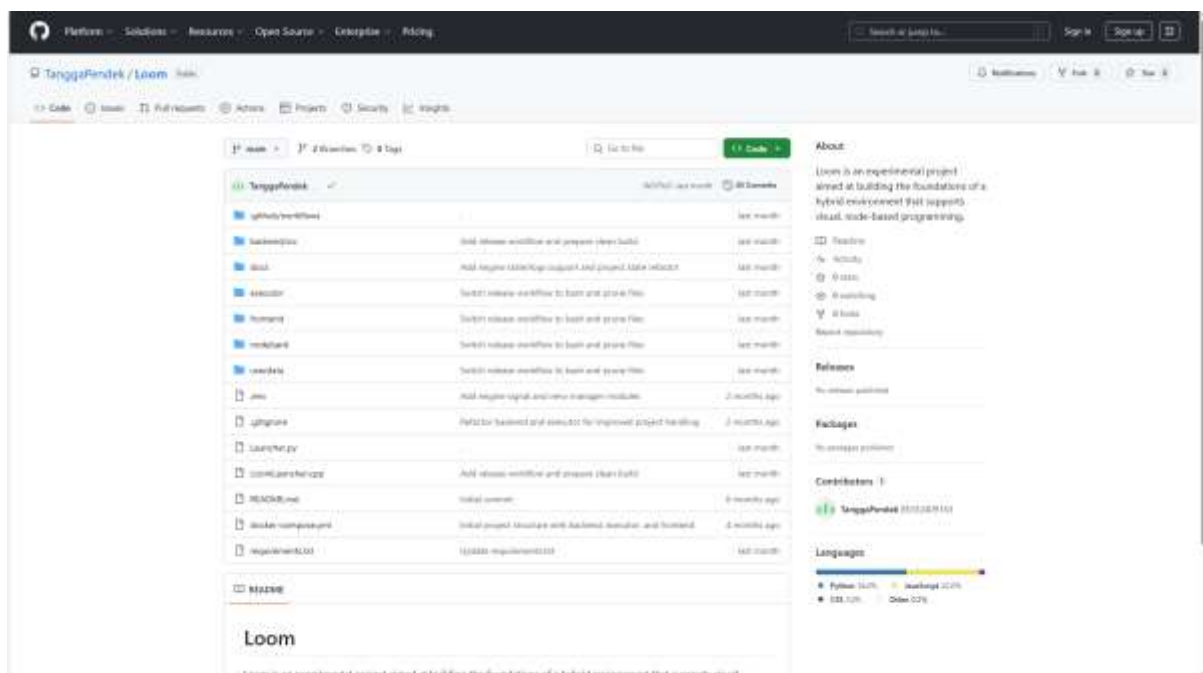


Figure 2: Github Page

5.2 Methodology

A hybrid development methodology was adopted, combining elements of structured planning and iterative development.

Hybrid Approach (Design-First + Agile)

The project began with a Design-First phase, where the initial focus was on research and system architecture. During this stage, existing tools such as node-based systems were studied to understand how logic flow and execution models are structured. This helped establish a clear blueprint before implementation began.

Following this, the project transitioned into an Agile-inspired iterative phase, where features were developed, tested, and refined in cycles.

This hybrid approach was chosen because:

- The system required a strong foundational design before implementation
- Iterative development allowed flexibility when integrating frontend and backend components

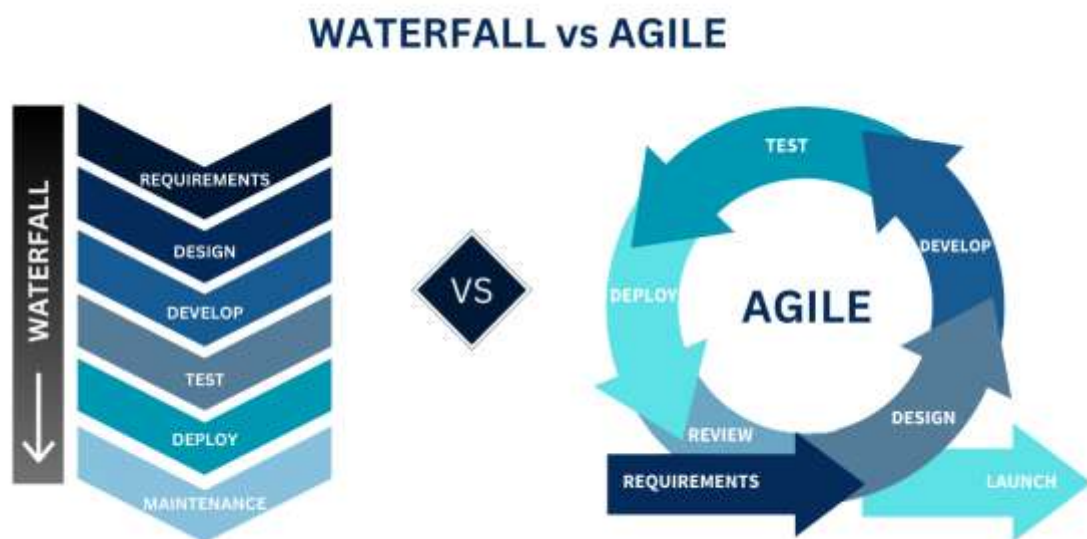


Figure 3: Waterfall vs Agile

Sprint Structure

Although a formal sprint management tool was not used, the development process followed a logical **sprint-based workflow** to structure tasks and monitor progress. Each sprint was time-boxed to approximately two weeks, with the following structure:

1. **Sprint Planning:**

- Tasks and features were prioritized based on importance and dependencies.
- Core backend functionality was treated as high priority, while UI and integration tasks were scheduled for subsequent sprints.

2. **Development Execution:**

- Features were implemented according to the sprint goal, with continuous testing and code refinement.
- Any required adjustments to previous components were made immediately to avoid compounding issues.

3. **Daily Review (Informal Standups):**

- As a single developer, progress was reviewed daily through self-assessment and task logging.
- Challenges or potential blockers were noted, and priorities were adjusted accordingly.

4. **Sprint Review:**

- At the end of each sprint, implemented features were tested to ensure they met functional requirements.
- Feedback from test cases or self-assessment guided the planning of subsequent sprints.

5. **Sprint Retrospective:**

- Reflections on what went well, what could be improved, and lessons learned were documented to inform future sprints.

This sprint structure ensured incremental progress while maintaining flexibility to adapt to unforeseen challenges, particularly in frontend-backend integration and feature refinement.

5.3 Monitoring and Control

Timeline and Milestones

The project was structured into two main development phases over a four-month period:

- **Months 1–2 (Core Development):**
Focused on backend and execution engine development. The primary goal was to ensure that Python scripts could be generated and executed correctly from structured input (JSON format). By the end of this phase, a functional core system was achieved.
- **Months 3–4 (Interface and Integration):**
Focused on frontend development and system integration. The user interface was built and connected to the backend, requiring iterative adjustments between components.

Handling Changes and Iteration

During development, multiple iterations were required, particularly when integrating frontend and backend systems. For example, certain frontend features required additional backend support, leading to repeated refinements of the execution engine.

This back-and-forth process reflects an Agile principle of continuous improvement., where changes were incorporated as new requirements emerged.

Time Management and Workload Distribution

As a single-developer project, workload distribution was managed individually. Time allocation for each sprint was guided by task complexity, dependencies, and priority:

- **High priority:** Core engine functionality, execution accuracy, and backend stability.
- **Medium priority:** UI components, frontend-backend synchronization, and feature refinement.
- **Lower priority:** Aesthetic enhancements, non-critical optimizations, and minor usability improvements.

Delays in lower-priority tasks did not compromise the overall system objectives. Regular informal reviews were conducted at the end of each sprint to evaluate progress, adjust future sprint goals, and identify risks early. This approach allowed effective control over the project timeline while providing flexibility for iterative improvements.

6.2 Phase 2: Design & Setup

During design, the system architecture was formalized, low-fidelity UI mockups were created, and the development environment was prepared. The architecture follows a layered design:

- **Frontend (React + ReactFlow):** A local facade providing instant feedback to the user while all actions are validated and synchronized with the backend.
- **Backend (FastAPI):** Authoritative controller of system state, persistence, and event validation.
- **Engine (Python Process):** Handles execution of node scripts dynamically, triggered by backend signals.

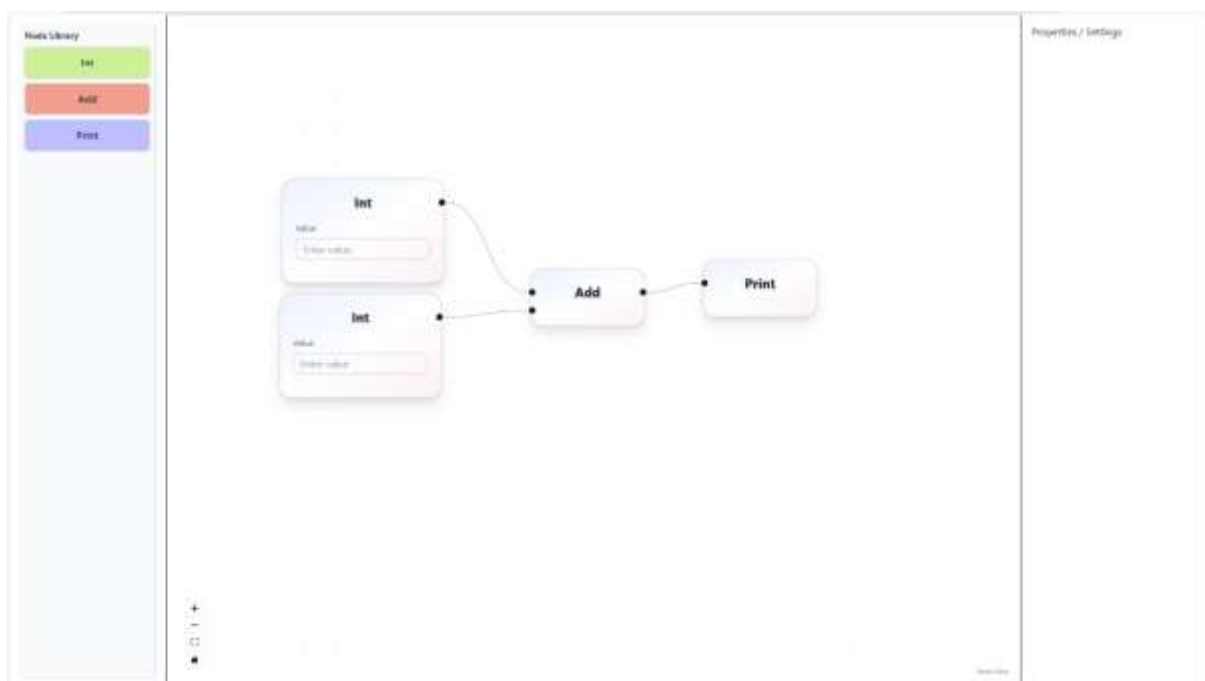


Figure 5 Frontend earlier mock up

A core design challenge was the graph execution order, requiring careful orchestration to maintain logical dependencies. Environment setup included Python dependencies, asynchronous communication setup, and establishing a persistent structure for node indexing. UI mockups guided node layout and interaction design, while ensuring usability and clarity for the target educational audience.

6.3 Phase 3: Implementation

The implementation phase of the LOOM project was a pivotal stage where design concepts were transformed into a working system. This phase focused not only on the development of core functionalities but also on the seamless integration of all system layers. Emphasis was placed on modularity, extensibility, and real-time feedback to ensure a robust and flexible node-based workflow engine. The implementation covered multiple subsystems, including the dynamic node loader, dependency-based scheduler, frontend integration, logging, WebSocket communication, virtual environment management, and runtime state management.

Dynamic Node Loader

A cornerstone of the LOOM architecture is the dynamic node loader, which empowers the engine to load and execute node scripts dynamically at runtime. This feature allows the system to support custom user-created nodes without requiring pre-registration or hardcoding into the engine. Node definitions are encapsulated in JSON format, containing all necessary metadata such as project identifiers, node types, connections, inputs/outputs, and additional configuration data. The backend orchestrates execution based on these definitions, respecting dependencies and ensuring correctness.

By leveraging runtime loading, the engine avoids bloating with unused built-in nodes, reducing memory consumption and improving startup performance. Moreover, this design allows users to extend the system organically, creating domain-specific workflows without modifying the core engine.

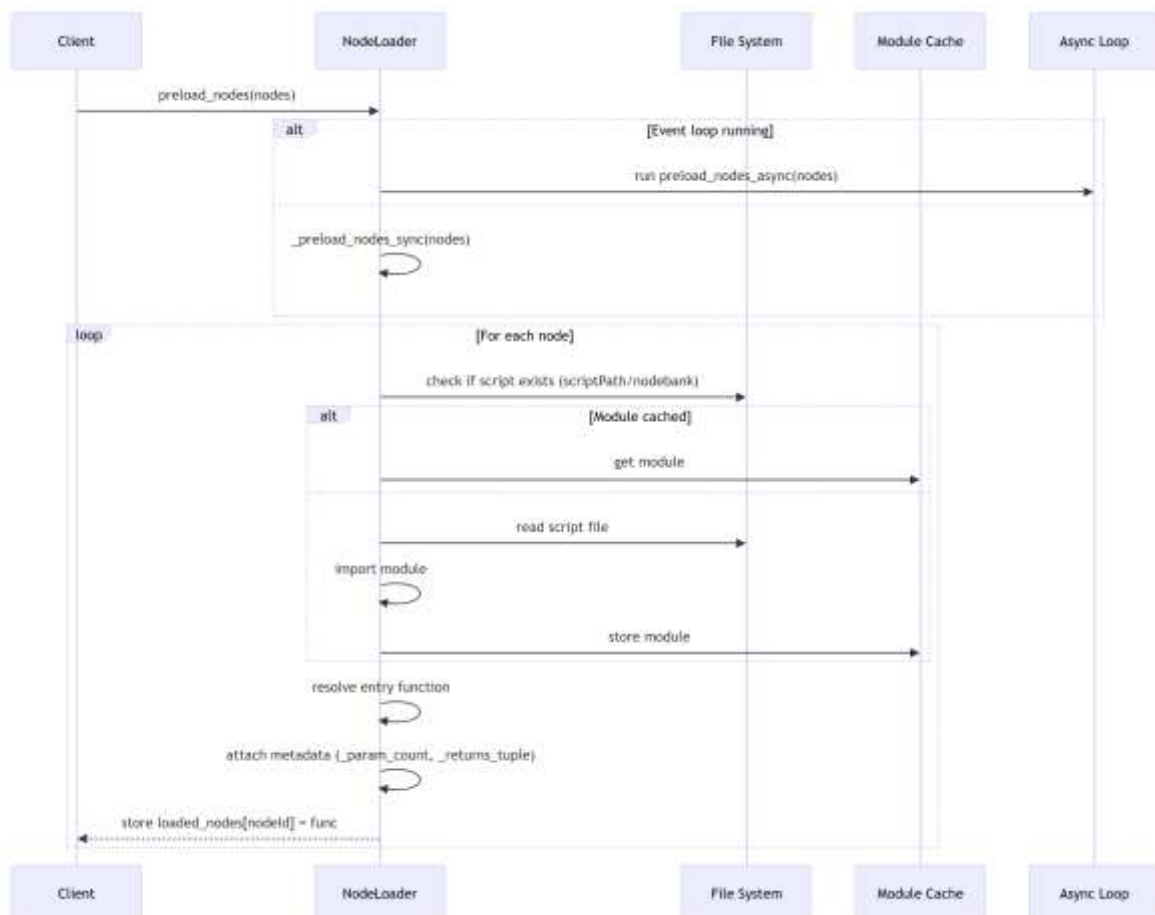


Figure 6: Dynamic Node Loader Sequence Diagram

Dependency-Based Scheduler

The scheduler is responsible for maintaining a consistent execution order among nodes. It operates on a dependency-based model, ensuring that:

- Nodes without inbound connections are queued for immediate execution.
- Completion of a node triggers updates to the dependent nodes' readiness status.
- Nodes are executed only after all required inputs are available.

This system guarantees predictable execution flows, even when nodes are loaded dynamically. The scheduler also supports error propagation, where failed nodes halt dependent processes and trigger logging mechanisms, preventing cascading failures. This layer abstracts away complexity from the user while ensuring the system remains deterministic, even in highly dynamic graphs.

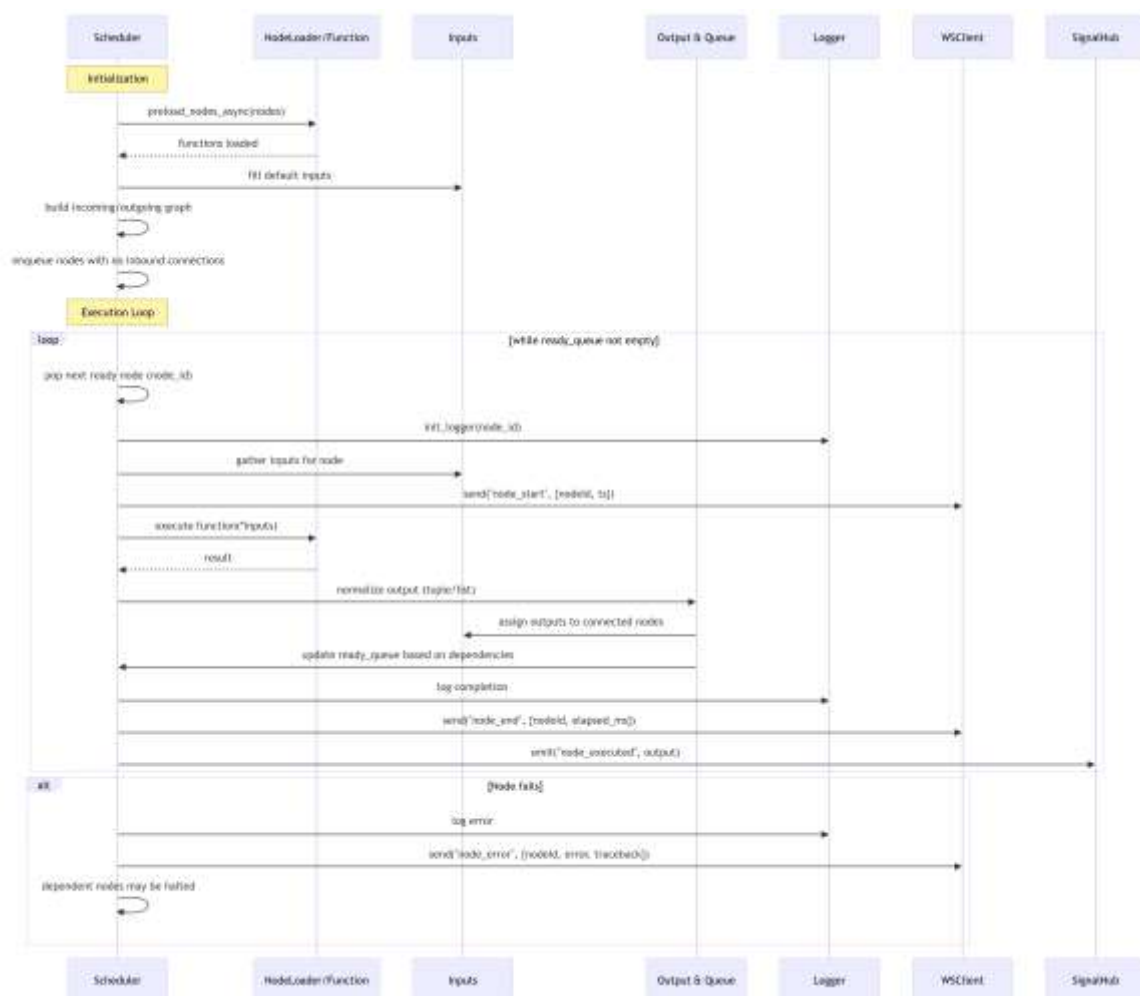


Figure 7: Main Execution Sequence Diagram

Frontend Integration

The frontend of LOOM serves as a responsive façade, reflecting user interactions immediately while maintaining backend authority over actual system states. Actions such as adding or moving nodes, creating connections, and editing parameters are first represented locally in the frontend interface and then validated and processed by the backend. This approach provides instant visual feedback, creating a smooth and interactive user experience.

To support real-time updates, a WebSocket layer was implemented, allowing the frontend to receive execution states, logs, and error notifications as the engine processes node graphs. This dual-layer system ensures that the engine maintains authoritative control while users perceive a responsive, near-instant interaction model.

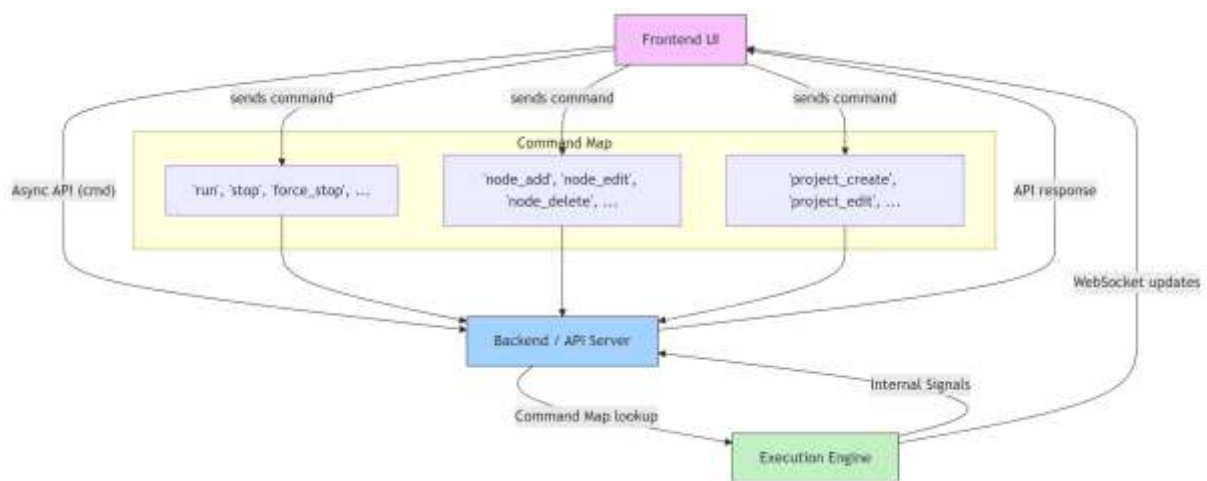


Figure 8: Loom Communication Diagram

Logging

Execution logs are critical for both debugging and system analysis. LOOM distinguishes between terminal outputs and frontend UI events. Logs are timestamped and structured to facilitate post-execution review. The logging system also integrates with the WebSocket broadcast mechanism to allow live monitoring, which is invaluable during testing and demonstration phases.

In addition to standard logging, an event tracking subsystem was implemented. This subsystem captures significant engine lifecycle events, including project initialization, execution start/stop, node errors, and completion signals. These events feed both the frontend and persistent logs, providing comprehensive insight into runtime behavior.

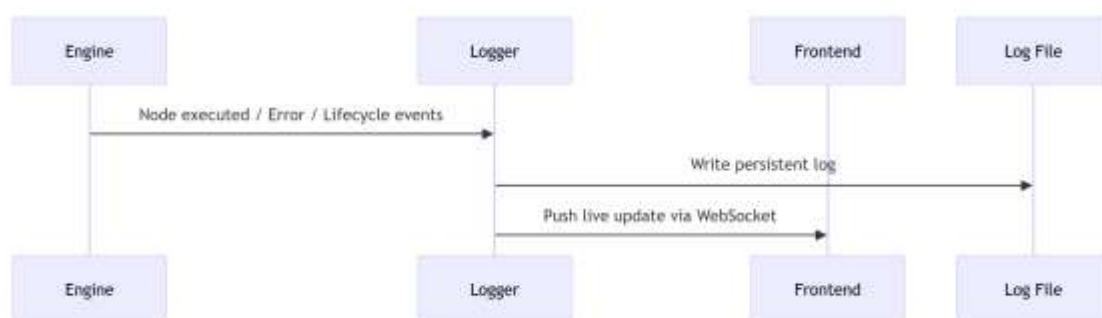


Figure 9: Logging Sequence Diagram

WebSocket Communication

To enable bidirectional, low-latency communication between frontend and backend, LOOM uses a WebSocket connection manager. This manager maintains a pool of active connections, handles client connection and disconnection events, and provides a thread-safe broadcast mechanism for engine states and output logs. It also periodically pings clients to maintain connection health and detect stale connections. This infrastructure ensures the frontend remains synchronized with the engine in real time, supporting interactive debugging and live monitoring of node executions.

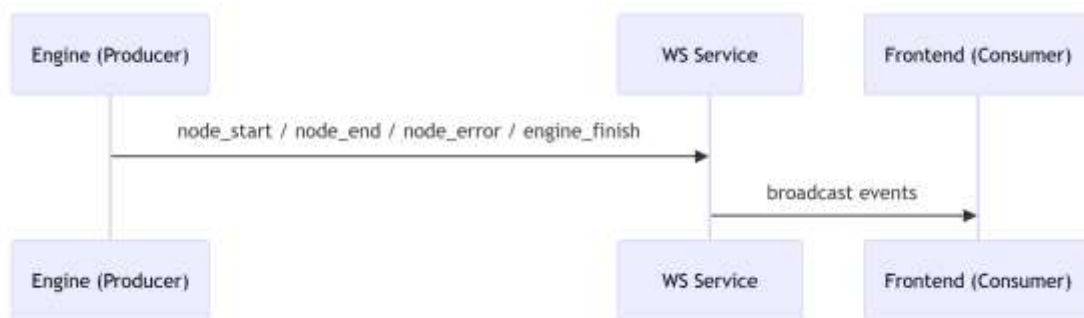


Figure 10: Websocket Sequence Diagram

Additional Features and Enhancements

During implementation, several auxiliary features were added to improve usability and system resilience:

- **Project Persistence:** Project graphs, node configurations, and execution logs are persisted in structured JSON files. This allows projects to be resumed or shared without manual export.
- **Node Indexing Service:** The index service maintains a searchable registry of available nodes, including custom user nodes. This improves discoverability and speeds up graph construction.
- **Frontend SPA Integration:** The system is bundled with a single-page application (SPA) frontend, providing an interactive drag-and-drop experience, with static assets served via FastAPI.

Overall, the implementation phase prioritized flexibility, reliability, and user experience. By combining dynamic node loading, dependency-aware scheduling, real-time frontend feedback, persistent logging, robust error handling, and modular subsystems, LOOM establishes a scalable foundation for complex node-based workflows and future feature expansions.

6.4 Phase 4: Testing & Refinement

Testing was conducted primarily through manual exploration of the system. Each feature was exercised by interacting with the engine directly, creating nodes, connecting them, and executing workflows to ensure expected behaviour. This self-testing approach focused on:

- **Feature Verification**

Each core feature such as dynamic node loading, dependency-based execution, and frontend-backend updates was explored in practice. For example, custom nodes were added and run to confirm that they executed without errors and that the results matched expectations.

- **Workflow Validation**

Complete graphs with multiple interconnected nodes were tested by running realistic scenarios. The dependency scheduler and dynamic node loader were observed to ensure nodes executed in the intended order and that outputs propagated correctly.

Logging & Debugging Refinements

During testing, the batch logging system was refined to provide clear and interpretable execution records. Logs differentiate between engine outputs, frontend events, and errors, allowing future users and developers to trace execution flow and diagnose issues effectively.

Design Decisions for Consistency

While real-time continuous streaming from backend to frontend was considered, the chosen backend-authoritative state with a frontend facade was maintained. This design ensures system integrity while providing responsive visual feedback, without requiring constant engine-to-frontend communication.

Through manual testing and iterative refinement, the system was confirmed to behave consistently under typical usage. The approach emphasized maintainable and predictable behavior over exhaustive automated testing, aligning with the educational purpose of LOOM as a visual programming tool.

6.5 Phase 5: Finalisation

The finalization phase consolidated development outputs and prepared LOOM for its intended use and demonstration. This phase included several key activities:

1. System Validation

Comprehensive validation ensured all core functionalities such as dynamic node loading, dependency execution, logging, and frontend-backend synchronization were working as designed. Test cases confirmed:

- Nodes executed in correct dependency order
- Custom and prebuilt nodes were handled reliably
- Error handling prevented crashes and maintained engine stability
- Frontend reflected backend state accurately
-

2. Documentation & Project Structuring

All code was annotated with inline comments and structured consistently to facilitate readability and maintainability. Node indexing structures were maintained to support discoverability, search, and future extensibility. Project directories and metadata followed a standardized format to enable reproducible workflows.

3. Report Writing & Reflection

Development decisions, technical challenges, and lessons learned were documented thoroughly. Reflection included:

- Trade-offs between real-time updates and system consistency
- The balance between modular extensibility and user accessibility
- Challenges in managing asynchronous node execution and state propagation
-

4. MVP Evaluation

The resulting Minimum Viable Product (MVP) demonstrates:

- Execution of dynamically loaded Python nodes with correct dependency handling
- Responsive and clear user interaction through a frontend interface
- Reliable logging and error reporting for debugging and learning
- Scalability potential for more complex educational or experimental workflows

6.6 Reflection and Lessons Learned

The development of LOOM provided valuable insights into the design and implementation of a modular, node-based visual workflow system. Throughout the project, several technical and conceptual challenges arose, particularly in balancing system flexibility with maintainability. Designing the node system itself required careful planning. Nodes had to be both dynamic and extendable, allowing users to create and execute custom scripts at runtime, while the engine needed to orchestrate execution flows correctly according to dependencies. Achieving this without compromising the separation of backend authority and frontend responsiveness was arguably the most significant technical challenge of the project.

During the iterative development process, several key challenges became evident:

1. **Dynamic Node Management**

Implementing a dynamic node loader that could handle arbitrary user-created nodes required extensive consideration of dependency resolution, error handling, and runtime execution safety. Ensuring that nodes executed in the correct order, that errors were caught and reported without crashing the engine, and that custom nodes could be added without modifying core code demanded careful architectural decisions.

2. **Frontend-Backend Synchronization**

Maintaining a responsive and accurate frontend while ensuring the backend retained authoritative state presented additional complexity. The chosen backend-authoritative model with a frontend facade allowed the frontend to provide immediate visual feedback without risking inconsistency. However, this approach required careful coordination of updates, particularly when nodes were executed dynamically or modified in real time. Manual testing revealed edge cases where the frontend state could lag behind the backend, leading to incremental refinements in signal handling and WebSocket broadcasting.

3. **Iterative Refinement and Self-Testing**

Without formal automated testing frameworks, the project relied heavily on manual testing and iterative refinement. Interacting directly with the engine, running different graph configurations, and observing outcomes was both a challenge and a learning opportunity. This approach allowed rapid identification of usability issues and unexpected behaviour.

The most rewarding moments occurred when the system operated cohesively. Nodes executed in the correct order, outputs propagated correctly, and the frontend accurately reflected the backend state. These successes highlighted the importance of clear modular design and reinforced the educational value of building a system that is both functional and extendable.

Lessons Learned and Future Considerations

Through this project, several broader lessons emerged:

- **Modular Architecture Matters**

Designing LOOM with modularity in mind. both in the engine and in node management allowed features to be extended with minimal disruption. This emphasizes that careful upfront planning in system architecture pays off when integrating new functionality later.

- **Iterative Development is Effective**

Even without formal testing frameworks, incremental self-testing and iterative development provided a reliable path to progress. Iterating over small functional improvements made it easier to isolate and address problems, particularly when dealing with complex dependency flows.

- **User Experience is Critical**

Balancing backend authority with frontend responsiveness taught the importance of interface design and user perception. Immediate visual feedback, clear error reporting, and intuitive node manipulation are essential, even in an educational prototype.

- **Planning for Scalability**

While the current MVP successfully handles basic educational workflows, future improvements could focus on scalability and real-time interactivity. WebSocket-based real-time updates could allow multiple users to interact with a shared graph simultaneously. Frontend enhancements, such as richer node visualization, grouping, and drag-and-drop improvements, would make the system more intuitive. Additionally, implementing more advanced logging and monitoring tools would support debugging and future feature expansions.

Chapter 7: Output Documentation

7.1 Output Diagrams

Architecture Diagram

The final system architecture is a hybrid desktop application that leverages pywebview to bridge a React-based frontend with a Python FastAPI backend. This allows for a native-feeling desktop experience with the power of Python's logic engine.

Key Components:

- **Frontend (React & React Flow):**
A node-based UI that handles the visual representation of the graph. It communicates with the backend via a standard REST API and a WebSocket layer for real-time execution feedback.
- **Backend (FastAPI):**
The core service orchestration layer. It utilizes a SignalHub to manage internal events and routes requests to specialized handlers:
 - **Graph Operations:**
Manages the CRUD logic for nodes and connections.
 - **Command and Control:**
Handles system initialization, state loading, and engine triggers (Run/Stop).
- **Managers & Services:**
 - **ExecutionManager:**
Coordinates with the Engine to process the logic flow.
 - **NodeManager & ProjectManager:**
Interface with the IndexService to resolve node types and project paths.
 - **StorageManager:**
Handles the file-based I/O for persistence.
- **Engine:**
A standalone Python process that executes the node scripts, utilizing a Node Loader to dynamically import functions from the NodeBanks (Built-in and Custom).

This architecture follows a modular and event-driven design, allowing independent components to communicate through defined interfaces while maintaining system scalability and flexibility.

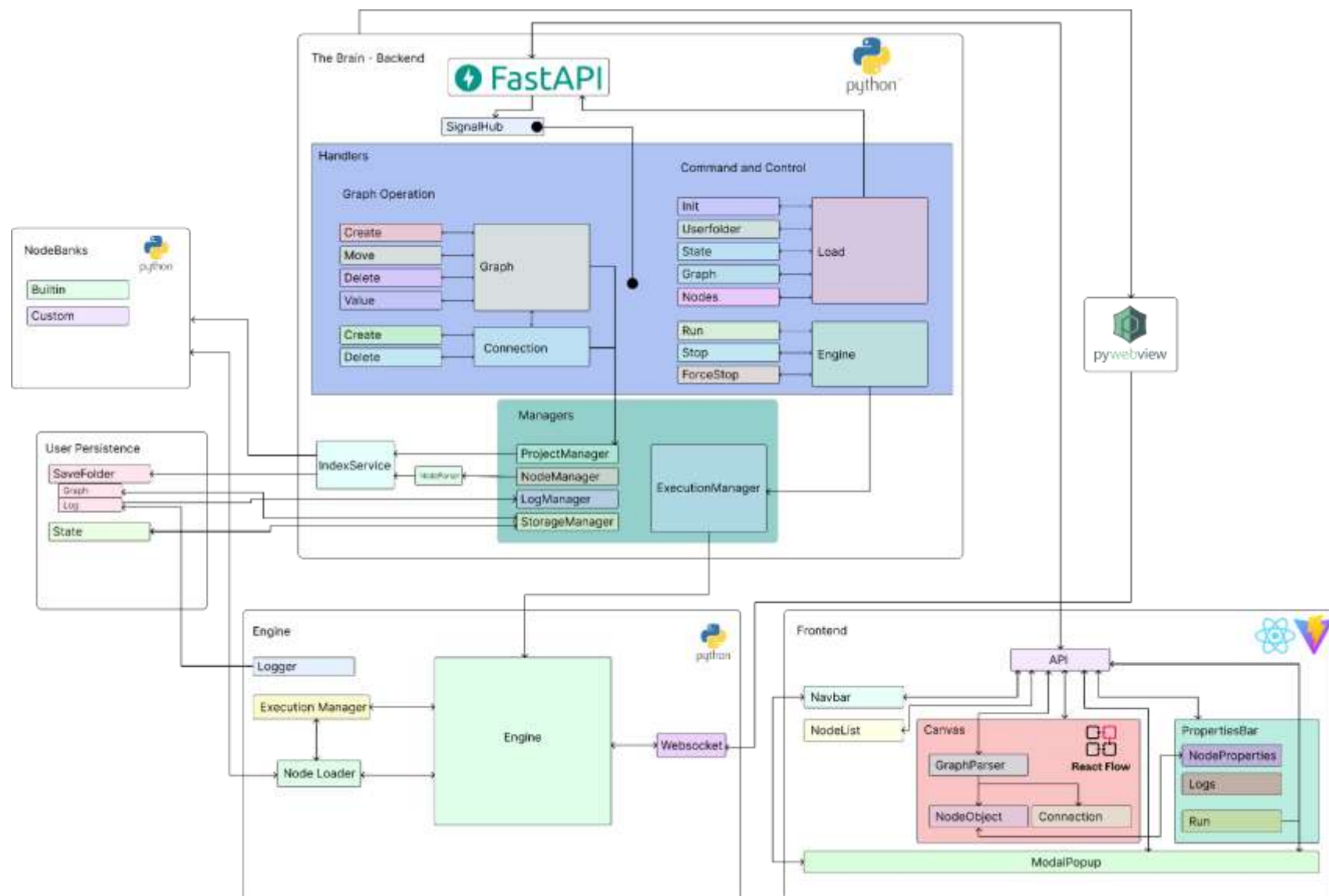


Figure 11: Loom System Architecture

Activity Diagram

The activity diagram illustrates the overall workflow of the system from the user's perspective. It captures key user actions such as project creation, selection, and deletion, as well as node manipulation processes including adding, editing, and positioning nodes within the graph.

Additionally, the diagram represents system-level operations such as refreshing the canvas and executing the graph logic. The execution process is shown as a transition from user-triggered actions to backend processing, culminating in the display of results.

This diagram provides a high-level understanding of how users interact with the system and how those interactions translate into system behavior.

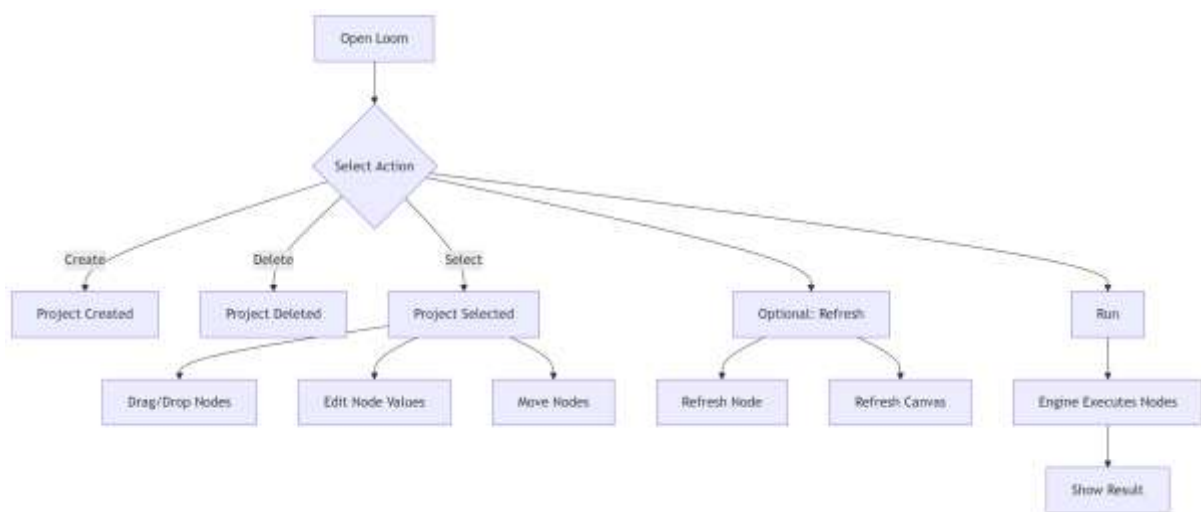


Figure 12: Loom Activity Diagram

Sequence Diagram

The sequence diagram demonstrates the interaction between the main system components during the execution process. It highlights the communication flow between the frontend interface, backend modules, and the execution engine.

When a user initiates execution, the request is processed by the execution module, which retrieves the current project state and validates the necessary resources. The system then emits an execution start signal and spawns the execution engine as a separate subprocess.

Parallel monitoring threads are created to observe the engine's standard output and error streams. These outputs are relayed through the internal SignalHub, enabling real-time feedback to the frontend. Upon completion, the system updates its execution state and notifies all relevant components.

This diagram emphasizes the asynchronous and event-driven nature of the system, particularly the use of real-time communication mechanisms.

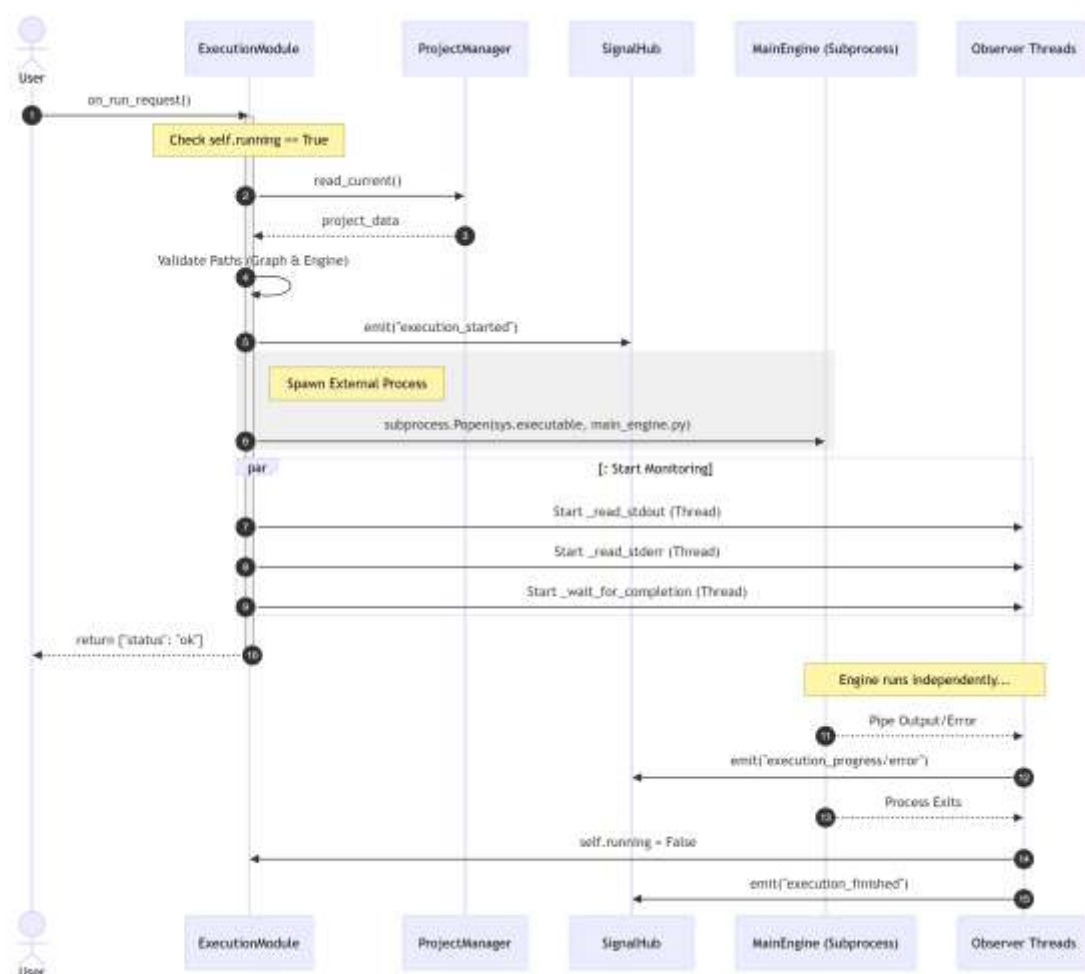


Figure 13: Loom Sequence Diagram

7.2 Database Structure

Database Design (Local Persistence)

The system eschews traditional SQL databases in favor of Local File-Based Persistence. This is structured around a centralized userdata directory.

The use of JSON as the primary data format enables structured, human-readable storage while maintaining flexibility for dynamic schema evolution. This is particularly suitable for representing graph-based data such as nodes and connections, where relationships are hierarchical and variable in size.

Data Schema (JSON):

- **savefile.json:**
The primary manifest for a project, containing metadata (author, timestamps), an array of node objects (positions, inputs, outputs), and a connection map.
- **index.json (Project & Node):**
Acts as a registry. The Project Index tracks all available local projects, while the Node Index maps node types to their entry functions and Python script paths.
- **state.json:**
Maintains the application's "memory," such as the last opened project and session history.
- **logs.json:**
A persistent buffer of execution messages, including timestamps and node-specific levels (info/error).

Design Rationale

- **Portability:** Projects can be shared by simply moving a folder; no database migration is required.
- **Transparency:** Users can inspect and manually edit savefile.json for debugging.
- **Performance:** Local I/O is sufficient for the scale of visual programming intended for students and beginners.

```

{
  "projectId": "",
  "projectName": "",
  "projectPath": "",
  "metadata": {
    "author": "",
    "description": "",
    "createdAt": "",
    "lastModified": ""
  },
  "nodes": [
    {
      "nodeId": "",
      "positionId": "",
      "name": "",
      "position": {
        "x": 0,
        "y": 0
      },
      "input": {
        "var": "Value",
        "value": ""
      },
      "output": {
        "string": ""
      },
      "ref": "",
      "scriptPath": "",
      "entryFunction": "",
      "metadata": {
        "operation": ""
      }
    }
  ],
  "connections": []
}

```

```

{
  "projectId": "proj_002_1654",
  "projectName": "Demo1",
  "projectPath": "G:\\Github\\Loon\\userdata\\Demo1\\savefile.json"
},
{
  "projectId": "proj_004_1156",
  "projectName": "Demo_Server",
  "projectPath": "G:\\Github\\Loon\\userdata\\Demo_Server\\saveFile.json"
},
{
  "projectId": "proj_005_1057",
  "projectName": "Demo_Visualizer",
  "projectPath": "E:\\Github\\Loon\\userdata\\Demo_Visualizer\\savefile.json"
}
]

```

```

{
  "nodeId": "io_input_node",
  "name": "Io Input",
  "type": "script_node",
  "scriptPath": "G:\\Github\\Loon\\nodebank\\builtIn\\io_input_node.py",
  "entryFunction": "io_input_node",
  "dynamic": {
    "inputs": [
      "Value"
    ],
    "outputs": [
      "Value"
    ]
  }
}

```

Figure 14: Json Format for project, projectindex and nodeindex

```

{
  "projectId": "proj_004_1156",
  "projectName": "Demo_Server",
  "lastModified": "2020-03-26T01:11:58.821543Z",
  "projectPath": "G:\\Github\\Loon\\userdata\\Demo_Server\\savefile.json",
  "history": [
    {
      "projectId": "proj_004_1156",
      "projectPath": "G:\\Github\\Loon\\userdata\\Demo_Server\\savefile.json"
    }
  ]
}

```

```

{
  "timestamp": "2020-03-26T00:30:02.751081Z",
  "nodeId": "node_13",
  "message": "Printing: {'path': '/hello', 'method': 'GET', 'status': 200, 'body': 'hello world'}",
  "level": "info"
},
{
  "timestamp": "2020-03-26T00:30:02.750058Z",
  "nodeId": "node_13",
  "message": "Printing: {'port': 8000, 'host': '0.0.0.0', 'cors': {'enabled': False, 'origins': '**', 'methods': ['GET', 'POST']}}",
  "level": "info"
}

```

Figure 15:JSON Format for state and logs

7.3 Completed Requirements

MVP Features Completed

The project successfully implemented all core features required for a functional minimum viable product (MVP). Users can create, modify, and delete projects, while nodes can be added, edited, and positioned within a graph-based structure that visually represents their relationships. The system includes a basic execution engine capable of processing node logic dynamically and a file-based persistence mechanism that allows projects to be saved and loaded reliably. These features collectively satisfy the minimum functional requirements, providing a fully operational node-based workflow environment.

Post-MVP Features (Implemented)

Beyond the MVP, several enhancements were integrated to improve usability, modularity, and performance. Real-time execution feedback is supported through WebSocket communication, while dynamic node loading from the NodeBank allows both built-in and custom nodes to be seamlessly integrated. The execution engine operates as an independent subprocess, improving stability and isolation from the frontend. Internally, a SignalHub-based event system facilitates modular communication between components, and real-time monitoring streams execution outputs (stdout/stderr) to the user interface. A node indexing system further supports dynamic registration and discovery of nodes, enabling the system to scale efficiently as more functionality is added.

Post-MVP Features (Future Work)

Planned enhancements aim to improve interactivity, clarity, and scalability. These include visual indicators highlighting active nodes during execution, refined UI/UX for better node connection comprehension, undo/redo functionality for graph edits, performance optimization for large workflows, and the addition of multi-user or collaborative editing capabilities. These improvements would further enhance the system's responsiveness and accessibility, particularly for complex workflows.

Objective Completion

All primary project objectives have been met. The system allows users to construct node-based workflows, execute logical processes dynamically, and maintain persistent project data. Post-MVP enhancements reinforce modularity, real-time feedback, and dynamic extensibility, demonstrating the system's readiness for advanced workflows and future expansion.

7.4 API and Command/Sync Mapping

The system employs a structured, command-based communication model between the frontend and backend, where each user operation corresponds to a dedicated signal or request key. This modular design improves maintainability and enables scalable extension of functionality. Operations are categorized across system lifecycle management, project and graph operations, node definitions, connection logic, and execution engine control.

Key characteristics of this design include:

- **Separation of concerns:** Each module handles its own commands and internal requests.
- **Execution flexibility:** Support for synchronous, asynchronous, or dual execution modes depending on operational requirements.
- **Event-driven architecture:** SignalHub facilitates responsive communication between frontend actions and backend processing.
- **Clear action mapping:** Distinction between user actions (e.g., adding a node) and backend processing requests ensures robust and predictable behavior.

For example, project creation triggers the asynchronous *project_create_request*, while graph node addition uses *project_node_add*. Execution engine operations, such as *engine_run_request* and *engine_stop_request*, are handled asynchronously, ensuring that long-running processes do not block the UI. Synchronous requests, such as *node_index_request* or *get_logs*, provide immediate feedback when required. This organized mapping ensures all interactions are traceable, maintainable, and easily extendable for future functionality.

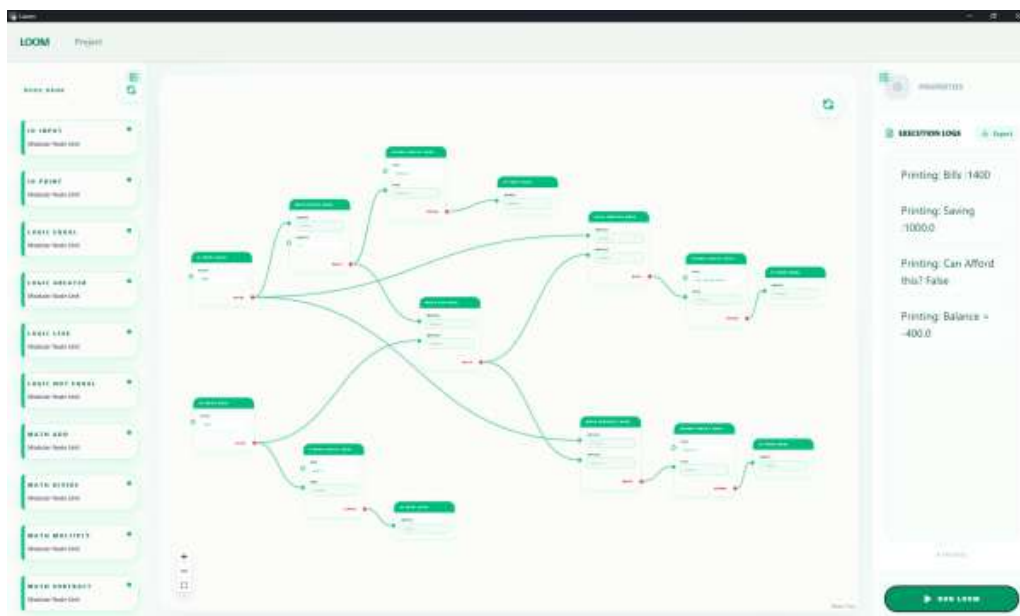


Figure 16: Loom Main Interface

COMMAND MAP

Frontend Command	Backend Signal / Request Key	Description
init	hot_reload_all	Hot reload all system nodes/projects
reload_projects	project_hot_reload	Reload all projects from storage
reload_nodes	node_hot_reload	Reload all node definitions
project_create	project_create_request	Create a new project
project_edit	project_update_request	Edit an existing project
project_delete	project_delete_request	Delete a project
project_load	project_load_request	Load a project into memory
load_graph	load_graph_request	Load the node graph for a project
graph_node_add	project_node_add	Add a node to the graph
graph_node_delete	project_node_delete	Delete a node from the graph
graph_node_update_input	graph_node_update_input_request	Update node input values
graph_node_edit	graph_node_move_request	Move a node in the graph
connection_create	connection_create_request	Add a connection between nodes
connection_delete	connection_delete_request	Remove a connection
node_add	node_create_request	Create a new node definition

node_edit	node_update_request	Edit node definition
node_delete	node_delete_request	Delete a node definition
node_move	node_move_request	Move node definition
run	engine_run_request	Execute the graph
stop	engine_stop_request	Stop execution
force_stop	engine_kill_request	Force stop execution
engine_get_state	engine_state_request	Get current engine state
engine_get_logs	engine_logs_request	Retrieve logs from backend

SYNC MAP

Frontend Command	Backend Signal / Request Key	Description
startup	startup_request	Initialize system synchronously
load_graph	load_graph_request	Fetch project graph synchronously
node_index	node_index_request	Fetch node index / registry
logs	get_logs	Retrieve log entries immediately

7.5 User Testing Results

Testing Description

User testing was conducted with a small group of participants familiar with basic computer interaction. They were asked to perform tasks including:

- Creating a project
- Adding, modifying, and connecting nodes
- Executing node-based processes

The purpose was to evaluate the system's usability, functionality, and correctness from an end-user perspective.

Results Summary

The testing indicated that:

- Users were able to understand and navigate the core functionality with minimal guidance.
- Node creation, modification, and execution worked reliably.
- Real-time updates and feedback significantly enhanced the responsiveness and usability of the system.

Key Findings

- Some users experienced minor confusion when managing node connections.
- The interface could benefit from clearer visual indicators for execution flow.
- Overall, participants were satisfied with the system's functionality and performance, but suggested improvements to the UI/UX for a more intuitive experience.

User Feedback Highlights

- Positive: The system allowed users to quickly construct and execute workflows, and the real-time feedback was appreciated.
- Areas for Improvement: Users suggested making node interactions and execution flow more visually clear, and improving the overall aesthetics and interface consistency.

Conclusion

The system meets its functional requirements and demonstrates effective performance for creating and executing node-based workflows. Future enhancements will focus on improving the UI/UX to further streamline user interaction and clarity.

USER ACCEPTANCE TEST

USER ACCEPTANCE TEST

Project Title: LOOM – Implementation of a Modular Node-Based Visual Workflow

Purpose: To evaluate system usability, functionality, and correctness from an end-user perspective

Participant Information

Name : _____
Background : _____
Date : _____

This User Acceptance Testing (UAT) session is conducted to assess the functionality and usability of the Graph-Based Execution System. The system allows users to create node-based workflows, connect components, and execute operations to produce outputs.

Participants are required to perform a series of tasks using the system and record their observations. The feedback collected will be used to improve system performance, usability, and reliability.

Instructions

1. Please complete each task listed in the Test Case Table.
2. After performing each task, record:
 - The actual result observed
 - Whether the test Passes or Fails
 - Any comments or issues encountered
3. If you are unsure how to proceed, attempt the task first before requesting assistance.
4. Provide honest feedback based on your experience using the system.

Figure 17: User Acceptance Test Page 1

USER ACCEPTANCE TEST

Test Case Table

Test ID	Task Description	Expected Result	Actual Result	Pass/Fail	Comments
TC01	Create a simple graph using available nodes and connect them correctly.	Nodes are successfully created and connected.			
TC02	Execute a graph with valid connections.	The system runs without errors and produces an output.			
TC03	Perform a basic operation using the provided nodes.	The system produces a correct and consistent result.			
TC04	Modify an existing graph (e.g., change connections or values) and execute again.	The system updates and produces a new correct result.			
TC05	Interact with the interface (add, move, and connect nodes).	The interface responds correctly without glitches or errors.			
TC06	Execute the same graph multiple times.	The system produces consistent results without crashing.			
TC07	Attempt to execute without completing all required connections.	The system prevents execution or does not produce a result.			

Figure 18: User Acceptance Test Page 2

USER ACCEPTANCE TEST

System Evaluation

Please rate the following aspects of the system:

Criteria	1	2	3	4	5
Ease of Use	☐	☐	☐	☐	☐
System Functionality	☐	☐	☐	☐	☐
Interface Design	☐	☐	☐	☐	☐
System Performance	☐	☐	☐	☐	☐
Overall Satisfaction	☐	☐	☐	☐	☐

*1 = Poor and 5 = Excellent

User Feedback

1. What did you like about the system?

2. What difficulties or issues did you encounter?

3. Suggestions for improvement:

Completion Acknowledgement

I confirm that I have completed the testing tasks and provided feedback based on my experience.

Signature: _____

Figure 19: User Acceptance Test Page 3

Chapter 8: End-Project Evaluation

8.1 Design Adherence

The system was developed with a “flow-first, syntax-second” philosophy, prioritizing visual understanding over strict code structure. The final implementation largely adhered to this principle but differed in several areas from the original design:

- **Architecture:** The addition of WebSocket communication for real-time feedback updates improved responsiveness, which was not part of the initial plan.
- **UI/UX:** While the original concept aimed for a fully polished interface, the final UI was simplified to focus on usability over aesthetics. Users could interact with nodes effectively, though visual clarity declined in complex graphs.
- **Technology:** Some initial plans, such as virtual environment isolation (venv) for dependency management, were dropped due to compatibility issues, simplifying the backend at the cost of flexibility.

Overall, these changes were justified to maintain stability, ensure correct node execution, and reduce unnecessary complexity. The core goal is to allow learners to understand programming flow visually will be remained central.

8.2 Objective Achievement

The objectives of the project were defined to focus on improving how users understand and construct program logic through a visual system. Based on the final implementation, the achievement of each objective is evaluated as follows:

Objective 1: To simplify programming workflow – Fully Achieved

The system successfully provides a drag-and-drop interface that reduces reliance on writing syntax directly. Users can construct workflows by connecting nodes, allowing them to focus on logical structure rather than code formatting. While custom scripts still require some understanding, the overall workflow is simplified compared to traditional coding approaches.

Objective 2: To improve understandability – Fully Achieved

The visual graph representation effectively demonstrates how data flows between nodes during execution. Users are able to observe the sequence of operations and understand how inputs and outputs propagate through the system. Although complex graphs may become visually dense, the system performs well within its intended scope of small to medium workflows.

Objective 3: To enhance modularity – Fully Achieved

The system supports modular design through reusable nodes. Users can define and reuse custom scripts as nodes, enabling consistent workflows across different projects. This approach encourages structured thinking and aligns with real-world software practices, where systems are built from smaller, independent components.

All objectives were achieved within the intended scope of the project. Minor limitations observed are related to usability and scale rather than the core functionality of the system.

8.3 System Evaluation

The system demonstrates several key strengths aligned with its educational purpose.

- **Clear Flow Representation**
The graph-based interface provides a straightforward way to understand execution flow. Users can visually trace how data moves between nodes, which is often difficult to grasp in traditional text-based code.
- **Simple Interaction Model**
Node connections remain consistent and predictable. The system follows clear rules for how nodes interact, making it easier for users to learn once they understand the basics.
- **Modular and Extendable Design**
Support for custom scripts allows users to extend the system beyond built-in functionality. This works well for guided learning scenarios and provides flexibility for more advanced users.
- **Lightweight and Focused Scope**
The system is intentionally designed for learning purposes. It avoids unnecessary complexity and focuses on delivering core functionality effectively.

Limitations:

- **User Interface Quality**
The most noticeable limitation is the UI. While functional, it lacks refinement and visual clarity, especially when working with multiple nodes.
- **Visual Clutter in Larger Graphs**
As the number of nodes increases, the canvas can become crowded, making it harder to follow the flow. This does not affect execution but impacts usability.
- **Learning Curve for Custom Nodes**
Although connections are simple, custom node usage requires understanding certain rules. New users may need initial guidance before fully utilising this feature.

Performance, Usability, and Reliability

- **Performance**

Performance is acceptable for the intended use case. The system handles small to medium workflows without major issues. Larger graphs may reduce responsiveness, but this is outside the primary scope.

- **Usability**

Usability is generally good after initial exposure. First-time users may require examples or guidance, but once familiar, interaction becomes straightforward.

- **Reliability**

The system is stable under normal use. Errors may occur when custom scripts return incorrect data or when connections are misconfigured, but these are expected within a flexible system.

8.4 Future Enhancements

Several improvements can be made to enhance the system further:

- **Major UI/UX Revamp**

A complete redesign of the interface would significantly improve clarity and user experience. Better layout management, visual hierarchy, and clearer execution indicators would reduce confusion, especially in larger graphs.

- **Improved Visual Feedback**

Enhancing how execution state is displayed (e.g., clearer node highlighting, progress indicators) would make the system easier to follow during runtime.

- **Better Guidance for Custom Nodes**

Adding documentation, templates, or onboarding examples would help users understand how to create and use custom nodes more effectively.

- **Graph Management Features**

Features such as grouping, collapsing nodes, or zoom-based detail control could reduce visual clutter in larger workflows.

- **Online Sharing / Library System**

Allowing users to share custom nodes or workflows could expand the system into a collaborative learning platform, where users can reuse and build upon each other's work

Chapter 9: Post-Mortem / Critical Reflection

9.1 Technology Review

The chosen stack (React frontend, FastAPI backend, Python execution engine) allowed rapid prototyping and flexibility.

Strengths included:

- Modular engine design that supports dynamic node loading.
- Real-time updates with WebSocket improving user experience.

Challenges included:

- Managing circular flows and complex dependencies.
- Iterative graph variants during development causing occasional confusion in execution order.

In retrospect, technologies chosen were suitable for the educational prototype. Future iterations may benefit from a stronger visualization library or built-in debugging tools.

9.2 Project Management Reflection

- Planning proved effective for core features but underestimated the complexity of advanced graph handling.
- Iterative development allowed flexibility, but some features required more time than initially estimated.
- Pre-development UI studies and testing could have prevented some user confusion.
- Key lesson: clearly define scope and complexity limits before implementation to avoid overengineering.

9.3 Personal Performance

Skills gained:

- System design and modular architecture planning.
- Problem-solving for asynchronous execution, dependency management, and frontend-backend synchronization.
- Debugging iterative and dynamic systems.

Challenges faced:

- Managing execution edge cases in custom nodes.
- Maintaining usability while implementing backend-authoritative design.

Areas for improvement:

- Pre-development research on UI/UX and testing methodologies.
- Time management for complex features.
- Exposure to collaborative workflows or real-world classroom integration.

9.4 User Testing Reflection

- Testing scope was limited but provided meaningful feedback on system clarity and responsiveness.
- Users initially struggled with node connections, though repeated trials improved comprehension.
- Feedback highlighted the need for better visual cues and simpler interfaces for non-expert users.

9.5 Ethical and Accessibility Reflection

The system is designed as a local, offline application, which naturally minimizes concerns related to data privacy and external security risks. Since all execution and data handling occur within the user's environment, users retain full control over their files and workflows. This design choice is appropriate for an educational tool, where simplicity and safety are prioritised over distributed or cloud-based features.

From an accessibility perspective, the system focuses on clarity and simplicity in interaction. The core drag-and-drop and node-connection model is straightforward and does not require complex input methods, making it generally approachable for new users. However, the current implementation does not yet include dedicated accessibility features such as keyboard navigation support or visual customization options. These can be considered for future improvement to broaden usability.

There are no significant ethical concerns in the current scope of the system. The application does not collect user data, does not rely on external services, and operates within a controlled environment. The use of custom scripts is aligned with its educational purpose, allowing users to experiment and learn within a flexible but self-contained system.

Overall, the system provides a safe and controlled environment suitable for learning. Future improvements can focus on enhancing accessibility and usability without compromising the simplicity of the current design.

9.6 Overall Reflection

The project successfully delivered a flow-first visual programming tool that emphasizes educational use. While improvements are possible for scalability, UI, and advanced features, the system demonstrates solid technical design, user-focused implementation, and modular extensibility.

Chapter 10: Conclusion

10.1 Project Summary

This project developed a modular, node-based visual programming environment designed specifically for learners who want to focus on understanding logical flow rather than memorizing syntax. By combining a lightweight frontend with a Python backend engine, the system supports dynamic node execution, real-time updates, and flexible project management. Users can create, connect, and configure nodes to visualize the execution of logic in a manner that closely resembles flowcharts, while still enabling the integration of advanced scripting capabilities for more experienced learners.

The architecture emphasizes modularity, allowing nodes to be added, modified, or removed without affecting the overall stability of the system. The system also includes features like file-based project saving, node metadata management, and execution history tracking, which together create a learning environment that is both interactive and persistent.

10.2 Achievement of Aim and Objectives

- Core aim achieved: enabling users to understand program logic visually.
- Objectives fully or partially achieved as outlined in Chapter 8, with key learning goals met.
- System supports both basic and advanced node scripts, allowing gradual skill progression for learners.

10.3 Contribution of the Project

Academic Contribution:

The project demonstrates a solid understanding of system architecture, modular design principles, and practical dependency management. It shows the application of modern programming paradigms in an educational context.

Practical Contribution:

The system provides a working prototype that could serve as a foundation for educational tools, plugin ecosystems, or AI-assisted learning environments. Educators and developers can extend it to create interactive tutorials, collaborative coding spaces, or experiment with real-time logic visualization.

Learning Outcomes:

Beyond technical skills in Python and frontend-backend integration, the project fosters problem-solving, modular thinking, and UX reflection. It encourages consideration of user experience in educational software design, highlighting the balance between functionality and usability.

10.4 Final Remarks

The project successfully delivers a functional, modular, and interactive educational tool. While the current system focuses on core functionality and usability, there is strong potential for further development. Enhancements could include:

- **UI/UX Improvements:** Making complex node graphs easier to navigate with better visual feedback and layout options.
- **Plugin Systems:** Enabling community-contributed nodes or AI-assisted suggestions to enrich the learning environment.
- **Cloud Integration:** Allowing users to save and share projects, fostering collaborative learning and access to shared resources.

With these extensions, the system could evolve into a scalable platform for cooperative learning, AI-assisted coding guidance, and community-driven content sharing. Ultimately, the project demonstrates how visual programming can lower the barrier to understanding computational logic while maintaining flexibility for advanced exploration. It lays the groundwork for future educational technologies that blend modular design, real-time interactivity, and learner-focused experience.

Reference

Blender Foundation (2019). *Home of the Blender Project - Free and Open 3D Creation Software*. [online] blender.org. Available at: <https://www.blender.org/> [Accessed 30 Mar. 2026].

docs.blender.org. (n.d.). *Geometry Nodes — Blender Manual*. [online] Available at: https://docs.blender.org/manual/en/latest/modeling/geometry_nodes/index.html [Accessed 30 Mar. 2026].

FastAPI (2023). *FastAPI*. [online] fastapi.tiangolo.com. Available at: <https://fastapi.tiangolo.com/> [Accessed 30 Mar. 2026].

GeeksforGeeks (2021). *Introduction to Visual Programming Language*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/software-engineering/introduction-to-visual-programming-language/> [Accessed 30 Mar. 2026].

Meta Open Source (2025). *React*. [online] react.dev. Available at: <https://react.dev/> [Accessed 30 Mar. 2026].

n8n.io. (n.d.). *n8n.io - a powerful workflow automation tool*. [online] Available at: <https://n8n.io/> [Accessed 30 Mar. 2026].

OpenJS Foundation (2019). *Node-RED*. [online] Nodered.org. Available at: <https://nodered.org/> [Accessed 30 Mar. 2026].

Pdp.gov.my. (2025). *Page Restricted*. [online] Available at: <https://www.pdp.gov.my/ppdpv1/akta/akta-pdppindaan-2024/> [Accessed 30 Mar. 2026].

Perlindungan Data Peribadi (PDP). (2025). *Akta Perlindungan Data Peribadi 2010 [Akta 709] • Perlindungan Data Peribadi*. [online] Available at: <https://www.pdp.gov.my/ppdpv1/akta/akta-pdp-2010-my/> [Accessed 30 Mar. 2026].

PYTHON (2025). *Python*. [online] Python.org. Available at: <https://www.python.org/> [Accessed 30 Mar. 2026].

Ray, P.P. (2017). A Survey on Visual Programming Languages in Internet of Things. *Scientific Programming*, 2017, pp.1–6. doi:<https://doi.org/10.1155/2017/1231430>.

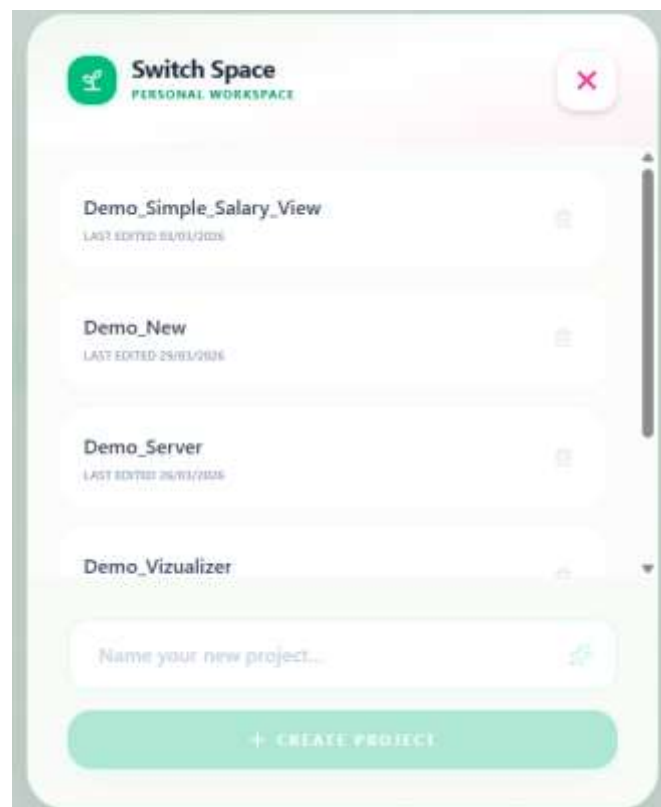
reactflow.dev. (n.d.). *React Flow - An open source library for building node based applications*. [online] Available at: <https://reactflow.dev/> [Accessed 30 Mar. 2026].

Retejs.org. (2018). *Rete.js - JavaScript framework for visual programming*. [online] Available at: <https://retejs.org/> [Accessed 30 Mar. 2026].

Vite (2024). *Vite*. [online] vitejs. Available at: <https://vite.dev/> [Accessed 30 Mar. 2026].

The screenshot displays the Loom Project interface. On the left, a sidebar titled "LOOM" and "Project" contains a list of test cases, each with a status icon and a title: "IS INPUT", "IS PRINT", "IS SORT", "IS MERGE", "IS INSERT", "IS SEARCH", "IS ADD", "IS DELETE", "IS UPDATE", "IS MAXIMIZE", and "IS MINIMIZE". The main area is a large, empty white space. On the right, a sidebar titled "EXECUTION LOG" shows a list of execution logs, each with a status icon and a title: "IS INPUT", "IS PRINT", "IS SORT", "IS MERGE", "IS INSERT", "IS SEARCH", "IS ADD", "IS DELETE", "IS UPDATE", "IS MAXIMIZE", and "IS MINIMIZE". At the bottom right, there is a green button labeled "RUN LOGS".

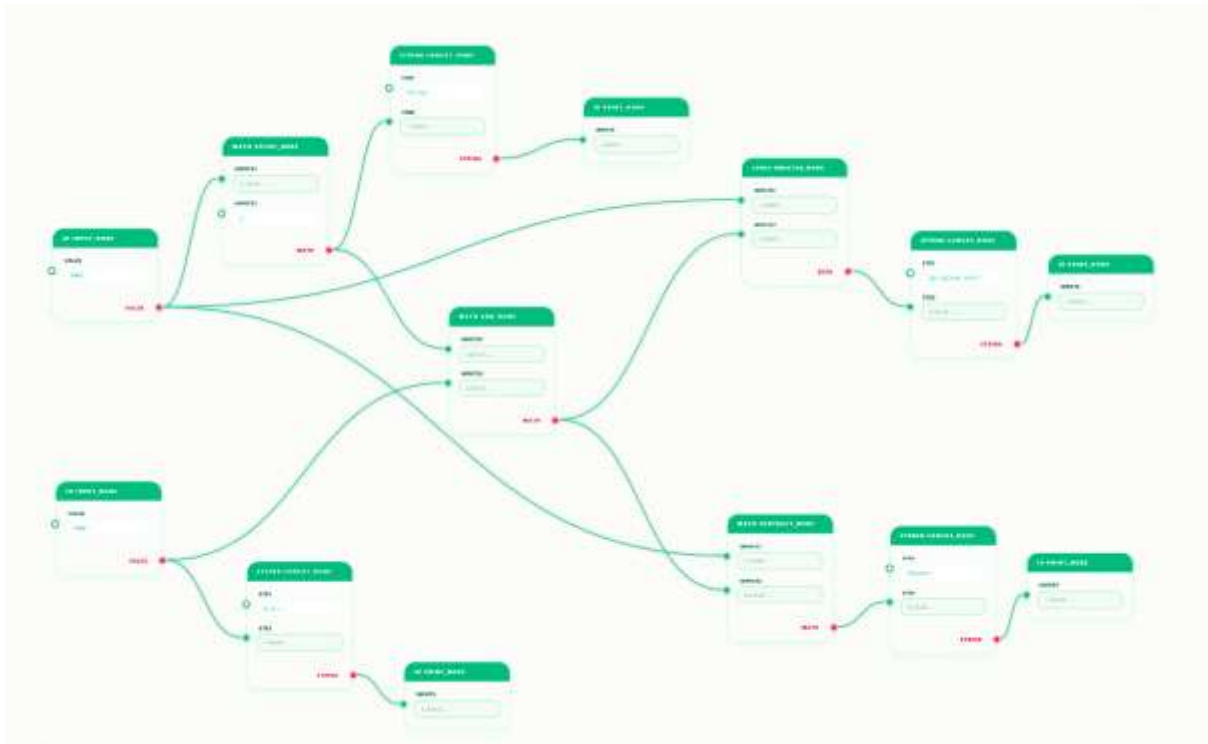
Appendix 1: Loom Main Interface



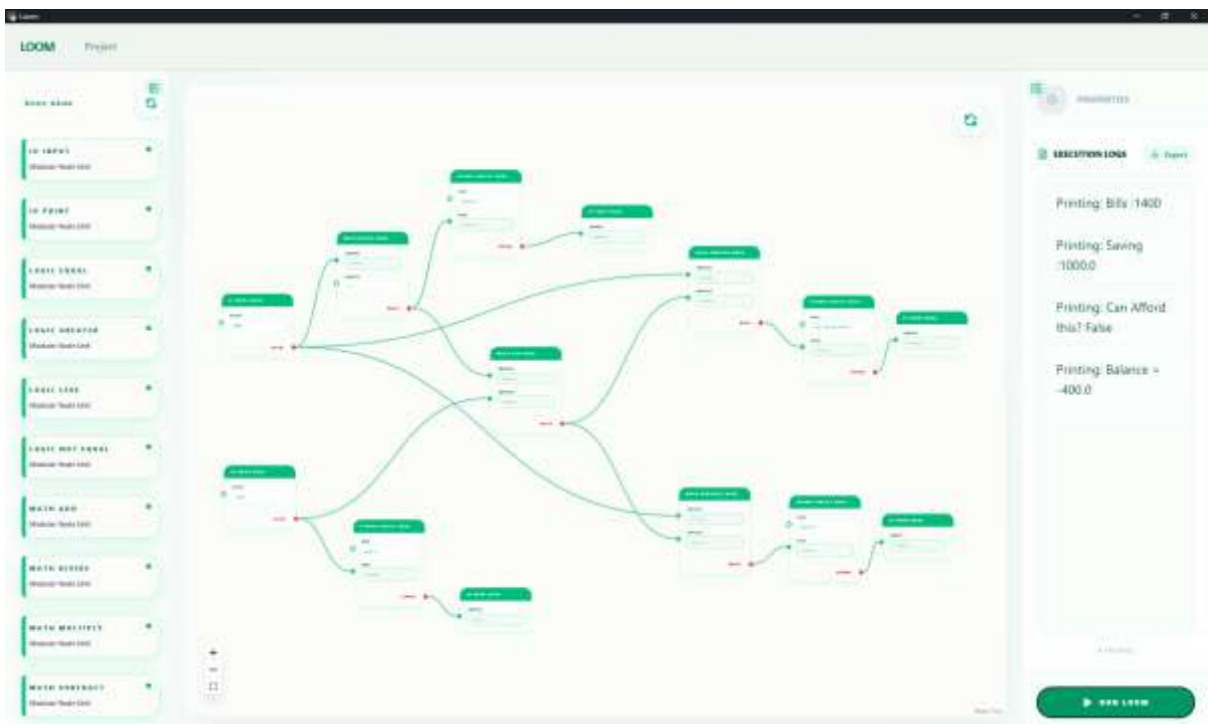
Appendix 2: Loom Project Selection Modal



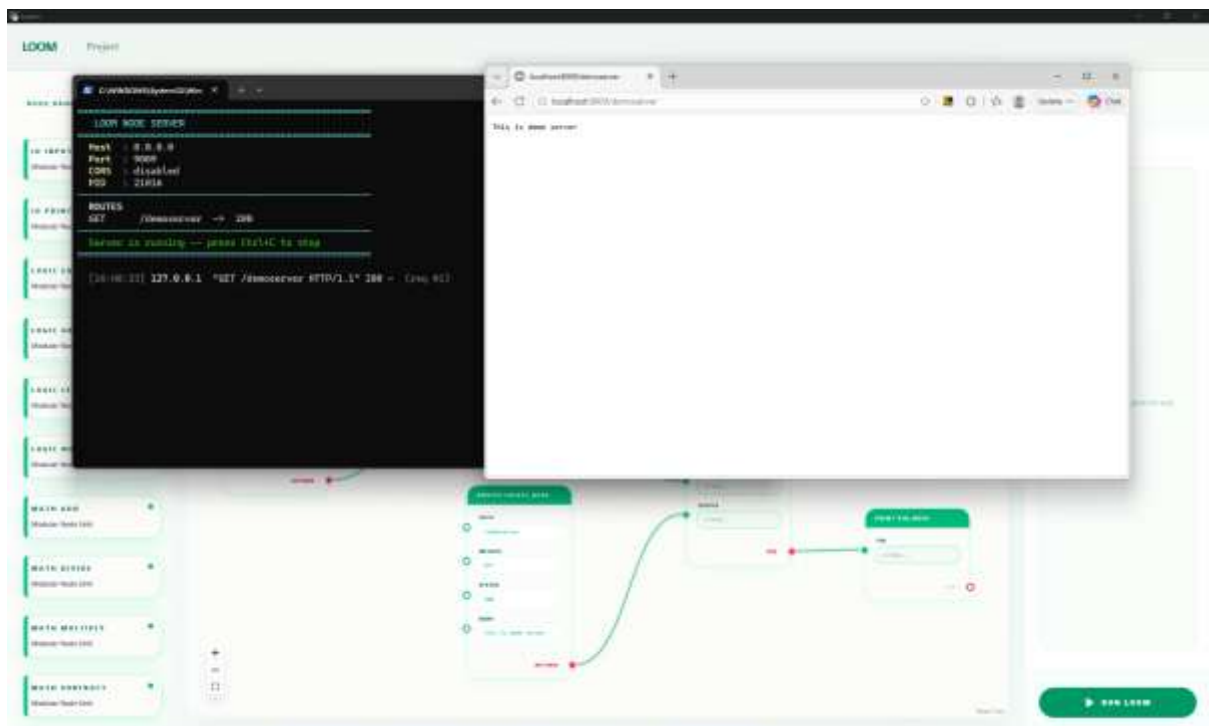
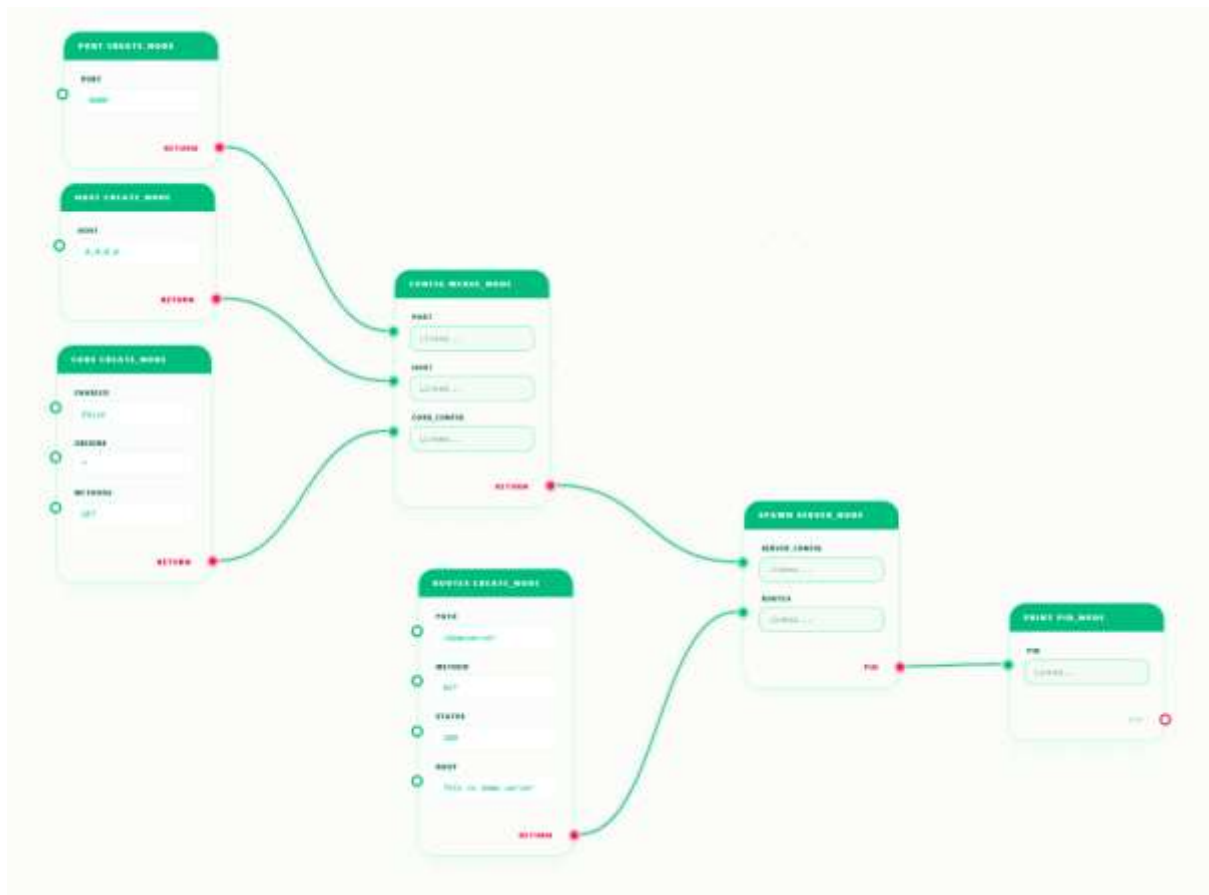
Appendix 3: Example of different size nodes

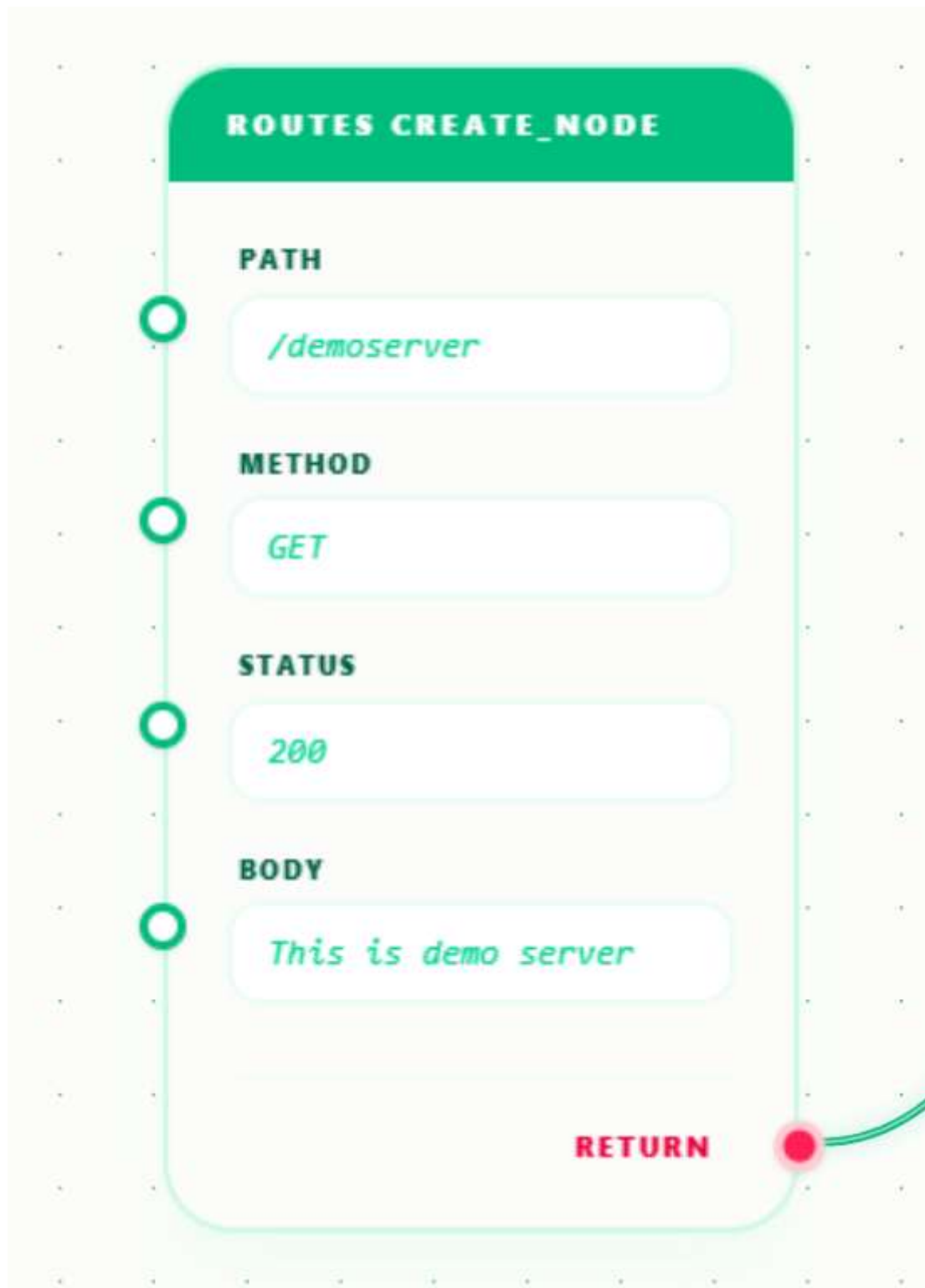


Appendix 4: Example of graph (Salary Calculator)

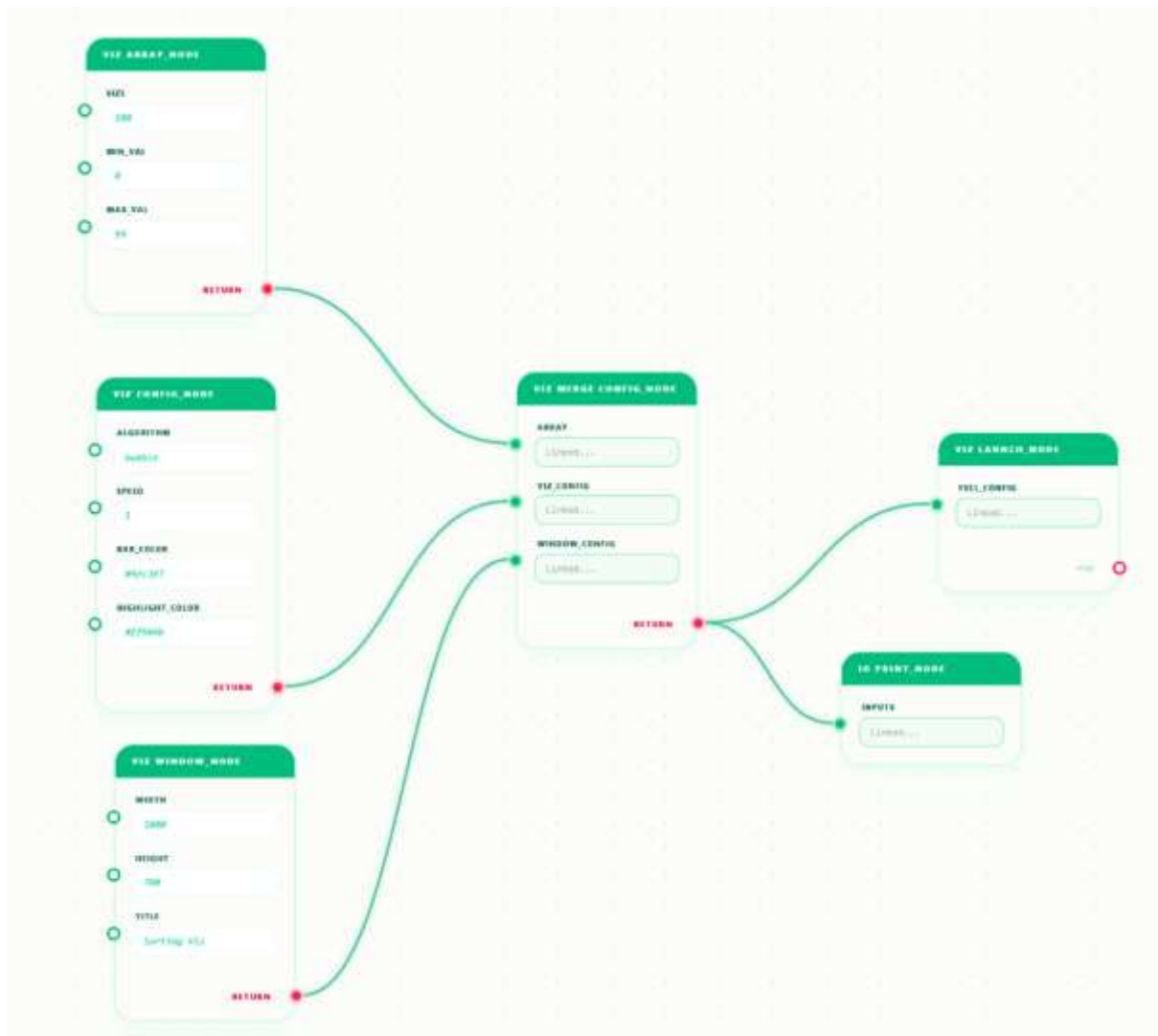


Appendix 5: Graph in full interface





Appendix 8: Routes node used in Server Demo



Appendix 9: Example of graph (Visualize Demo)



EXECUTION LOGS

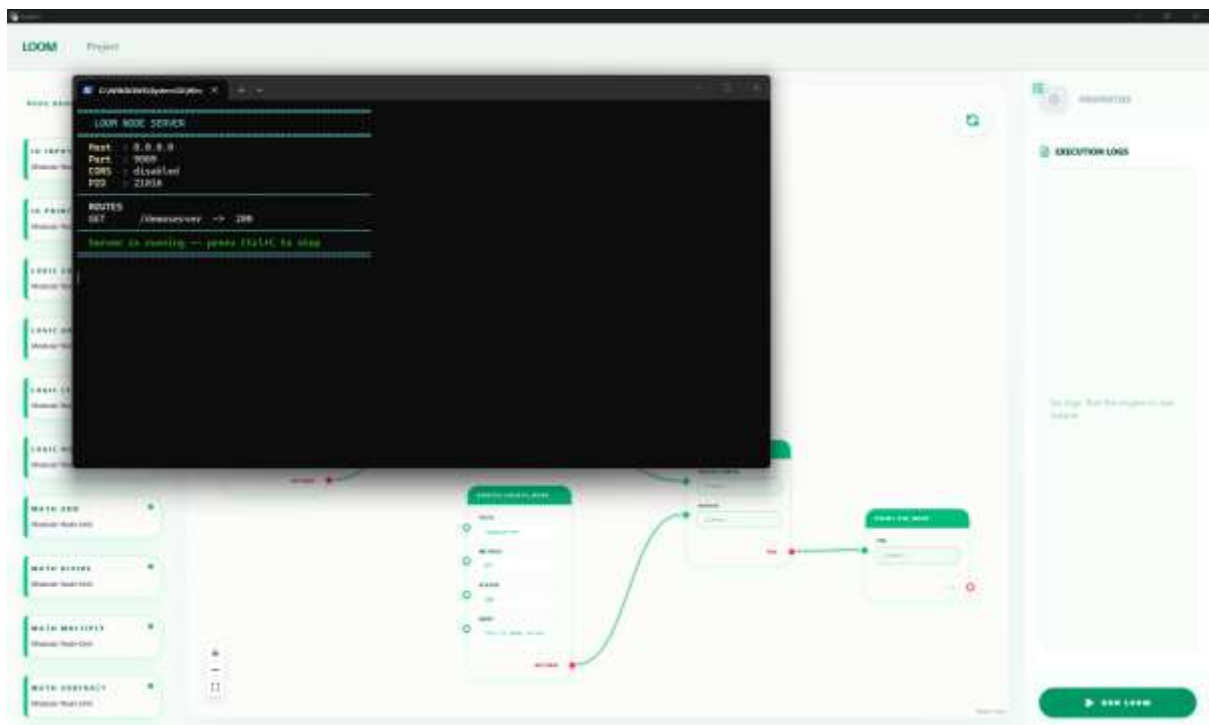
 Export

```
13, 22, 40, 82, 61, 0,  
32, 55, 36, 42, 0, 28,  
46, 51, 95, 74, 87,  
92, 39, 79, 31, 12,  
60, 90, 54, 42, 26, 2,  
81, 67, 16, 98, 20,  
62, 14, 70, 31, 55,  
86, 13, 20, 64, 20,  
30, 59, 71, 11, 14,  
87, 41, 5, 88, 10, 29,  
39, 13, 75, 43, 20,  
46, 71, 37, 51], 'viz':  
{ 'algorithm':  
'bubble', 'speed': 1,  
'bar_color':  
'#4fc3f7',  
'highlight_color':  
'#ff6b6b'},  
'window': { 'width':  
1000, 'height': 700,  
'title': 'Sorting Viz'}}
```

Appendix 10: Visualizer Demo log print

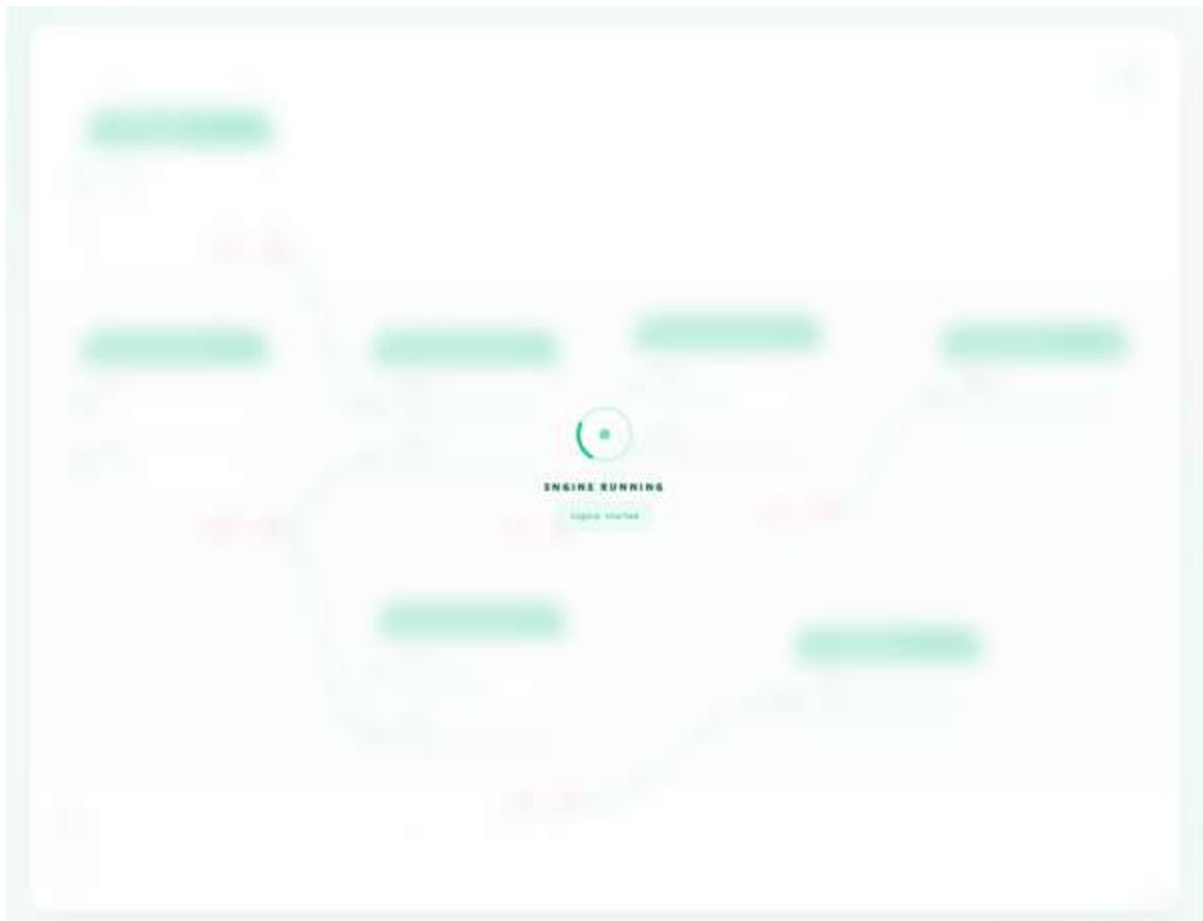


Appendix 11: Visualizer Demo running

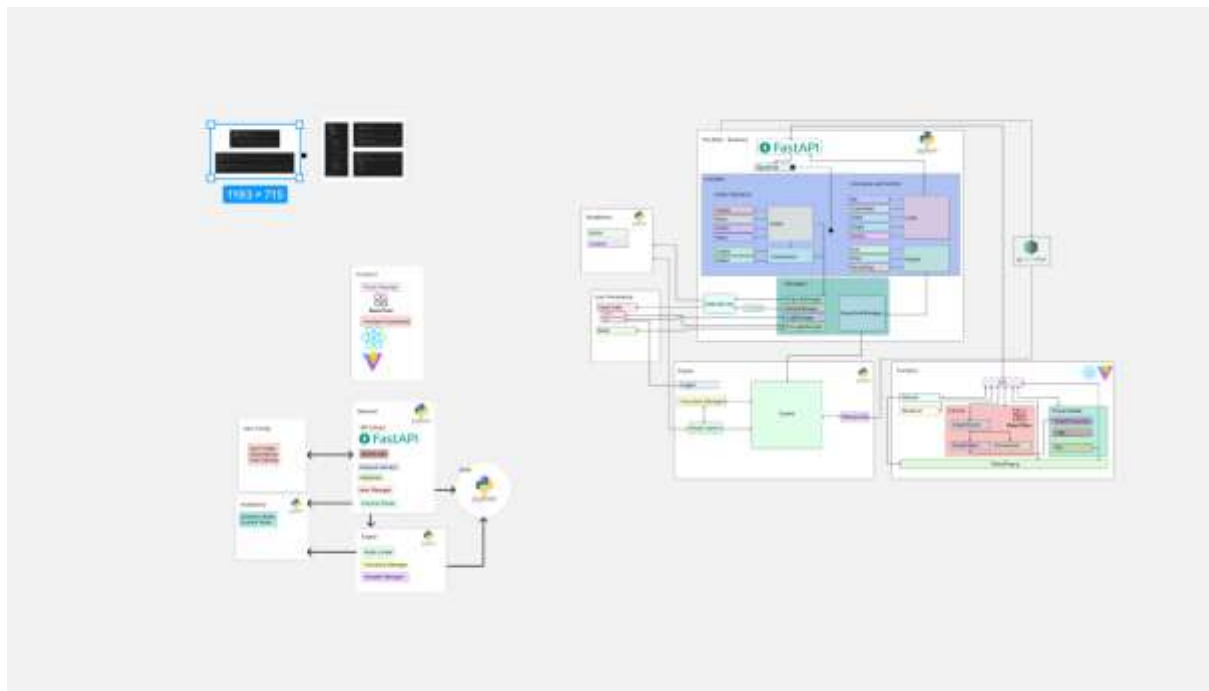


```
C:\Users\ManggaMasam\AppData\Local\Microsoft\Windows\Terminal
CMD RECEIVED: engine_get_logs
INFO: 127.0.0.1:51237 - "POST /dispatch HTTP/1.1" 200 OK
[ENGINE] WS service started (pid=23884)
[WS-SERVICE] Starting on ws://localhost:8001
[WS-SERVICE] Engine -> ws://localhost:8001/engine
[WS-SERVICE] Frontend -> ws://localhost:8001/frontend
[WS-CLIENT] Connected to ws://localhost:8001/engine
[ENGINE] Initializing...
[ENGINE] Running graph...
[ENGINE] >> Running node: 'viz config_node' (node_1)
[ENGINE] OK Node 'viz config_node' finished (0ms)
[ENGINE] >> Running node: 'viz window_node' (node_2)
[ENGINE] OK Node 'viz window_node' finished (0ms)
[ENGINE] >> Running node: 'viz array_node' (node_3)
[ENGINE] OK Node 'viz array_node' finished (0ms)
[ENGINE] >> Running node: 'viz merge config_node' (node_4)
[ENGINE] OK Node 'viz merge config_node' finished (0ms)
[ENGINE] >> Running node: 'viz launch_node' (node_5)
[launch_sorter] pid=23332 algorithm=bubble array_size=100
[ENGINE] OK Node 'viz launch_node' finished (8ms)
[ENGINE] >> Running node: 'io print_node' (node_6)
{'array': [93, 40, 31, 29, 85, 70, 64, 95, 41, 16, 15, 31, 33, 18, 79, 79, 41, 92, 10, 7, 25, 64, 97, 52, 75, 77, 8, 55, 12, 59, 85, 9, 66, 63, 91, 47, 55, 78, 99, 61, 65, 94, 81, 90, 68, 48, 18, 64, 71, 61, 85, 13, 18, 70, 15, 63, 91, 70, 35, 99, 15, 62, 48, 48, 63, 53, 97, 7, 78, 26, 88, 81, 34, 96, 38, 56, 12, 63, 2, 42, 21, 32, 58, 18, 16, 62, 72, 46, 98, 76, 85, 39, 83, 92, 3, 16, 9, 33, 52, 38], 'viz': {'algorithm': 'bubble', 'speed': 1, 'bar_color': '#4fc3f7', 'highlight_color': '#ff6b6b'}, 'window': {'width': 1000, 'height': 700, 'title': 'Sorting Viz'}}
[ENGINE] OK Node 'io print_node' finished (1ms)
[ENGINE] Execution completed successfully
[ENGINE] WS service stopped
[BACKEND] Engine process exited (code=0)
```

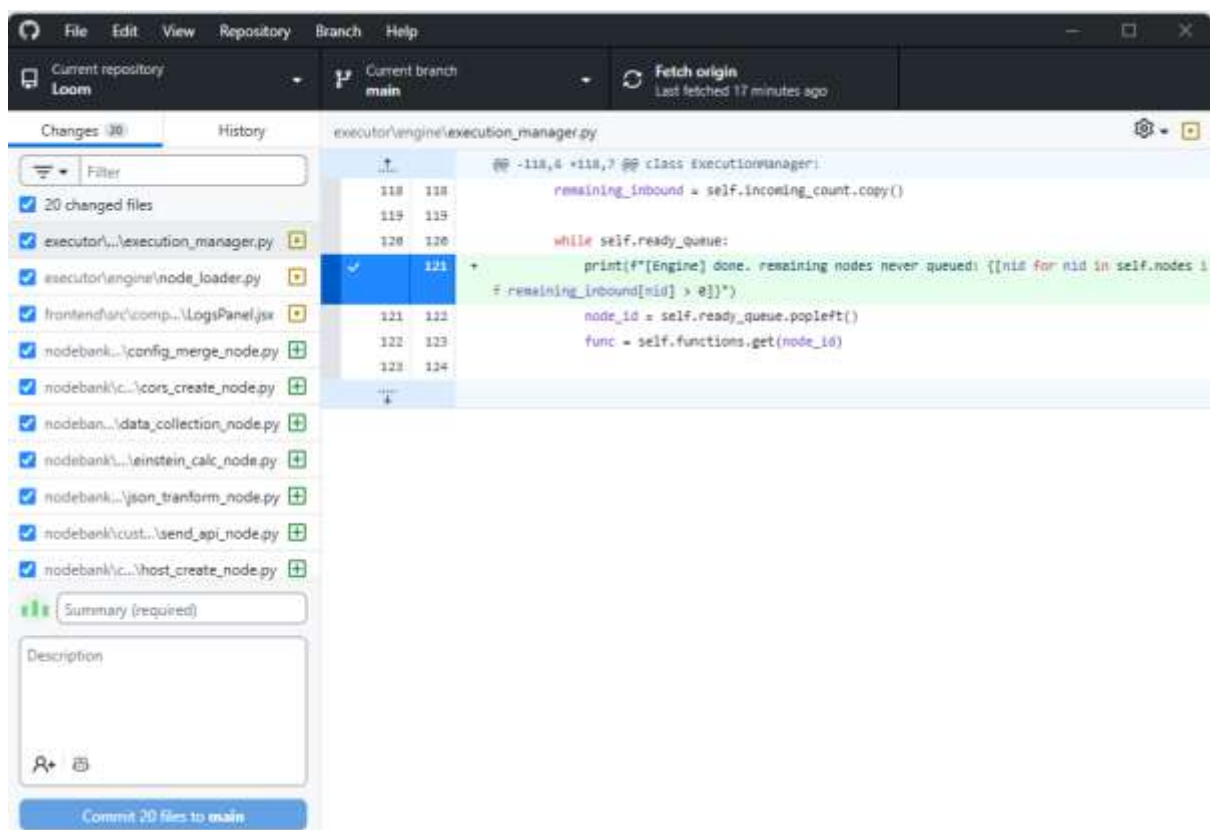
Appendix 12: Example of running terminal



Appendix 13: Canvas Execution Overlay



Appendix 14: Figma system architecture iteration



Appendix 15: Github Desktop of uncommit changes

